Red Hat Enterprise Linux for SAP Solutions 9

Deploying SAP HANA Scale-Up System Replication High Availability

Creating an HA cluster for automating scale-up HANA system replication with the \"Advanced Next Generation Interface\" SAP HANA resource agents (\"angi\")

Last Updated: 2025-09-11

Red Hat Enterprise Linux for SAP Solutions 9 Deploying SAP HANA Scale-Up System Replication High Availability

Creating an HA cluster for automating scale-up HANA system replication with the \"Advanced Next Generation Interface\" SAP HANA resource agents (\"angi\")

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at
http://creativecommons.org/licenses/by-sa/3.0/
. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

Java ® is a registered trademark of Oracle and/or its affiliates.

XFS ® is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL ® is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js ® is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack ® Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

Configure SAP HANA system replication for your scale-up environment and install a Pacemaker cluster. Manage your HANA instance more efficiently by using the RHEL for SAP Solutions components for higher availability.

Table of Contents

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION 4

CHAPTER 1. INTRODUCTION TO SAP HANA SCALE-UP SYSTEM REPLICATION HA 5
1.1. TERMINOLOGY 5
1.2. PERFORMANCE-OPTIMIZED SAP HANA SCALE-UP HA 5
1.3. CLUSTER RESOURCE AGENTS AND TOOLS FOR SAP HANA HA 6
1.4. SAP HANA HA/DR PROVIDER HOOKS 7
1.5. SUPPORT POLICIES FOR SAP HANA HIGH AVAILABILITY 9

CHAPTER 2. PLANNING THE HA CLUSTER SETUP 10
2.1. SUBSCRIPTION AND REPOSITORIES FOR SAP HANA HA 10
2.2. OPERATING SYSTEM REQUIREMENTS 10
2.3. STORAGE REQUIREMENTS 11
2.4. NETWORK REQUIREMENTS 11
2.5. HA CLUSTER REQUIREMENTS 11
2.6. SAP HANA PLANNING 12

CHAPTER 3. INSTALLING SAP HANA FOR A 2-NODE HA CLUSTER SETUP 14
3.1. INSTALLING A SCALE-UP SAP HANA INSTANCE 14
3.2. DISABLING SAP HANA INSTANCE AUTOSTART 15

CHAPTER 4. CONFIGURING THE SAP HANA SYSTEM REPLICATION 17
4.1. PREREQUISITES FOR THE SAP HANA SYSTEM REPLICATION SETUP 17
4.2. PERFORMING AN INITIAL HANA INSTANCE BACKUP 17
4.3. CONFIGURING THE PRIMARY HANA REPLICATION INSTANCE 20
4.4. CONFIGURING THE SECONDARY HANA REPLICATION INSTANCE 20
4.5. TESTING THE HANA SYSTEM REPLICATION 23

CHAPTER 5. CONFIGURING THE PACEMAKER CLUSTER 25
5.1. DEPLOYING THE BASIC CLUSTER CONFIGURATION 25
5.2. CONFIGURING GENERAL CLUSTER PROPERTIES 27
5.3. CONFIGURING THE SYSTEMD-BASED SAP STARTUP FRAMEWORK 28
5.4. INSTALLING THE SAP HANA HA COMPONENTS 29
5.5. CONFIGURING THE HANASR HA/DR PROVIDER FOR THE SRCONNECTIONCHANGED() HOOK METHOD 30
5.6. CONFIGURING THE CHKSRV HA/DR PROVIDER FOR THE SRSERVICESTATECHANGED() HOOK METHOD 33
5.7. CREATING THE HANA CLUSTER RESOURCES 34
5.8. CREATING THE VIRTUAL IP RESOURCE 39
5.9. ADDING A SECONDARY (READ-ENABLED) VIRTUAL IP ADDRESS 41

CHAPTER 6. TESTING THE SETUP 43
6.1. DETECTING THE SYSTEM REPLICATION STATE CHANGES 43
6.2. TRIGGERING THE INDEXSERVER CRASH RECOVERY 45
6.3. TRIGGERING A HANA TAKEOVER USING CLUSTER COMMANDS 47
6.4. TRIGGERING THE SAPHANAFILESYSTEM FAILURE ACTION 48
6.5. CRASHING THE NODE WITH THE PRIMARY INSTANCE 49
6.6. CRASHING THE NODE WITH THE SECONDARY INSTANCE 50
6.7. STOPPING THE PRIMARY INSTANCE USING SAP COMMANDS 51
6.8. STOPPING THE SECONDARY INSTANCE USING SAP COMMANDS 52

CHAPTER 7. FINISHING THE SETUP 53
7.1. ENABLING THE AUTOMATIC REGISTRATION OF HANA AFTER A TAKEOVER 53

7.2. REVIEWING THE FINAL CLUSTER STATE 54

CHAPTER 8. MAINTENANCE PROCEDURES 57
8.1. CLEANING UP THE FAILURE HISTORY 57
8.2. TRIGGERING A HANA TAKEOVER USING CLUSTER COMMANDS 57
8.3. UPDATING THE OPERATING SYSTEM AND HA CLUSTER COMPONENTS 57
8.4. PERFORMING MAINTENANCE ON THE SAP HANA INSTANCES 58
8.5. REGISTERING THE FORMER PRIMARY AFTER A TAKEOVER 61

CHAPTER 9. TROUBLESHOOTING 63
9.1. THE SRHOOK CLUSTER ATTRIBUTE VALUE IS INCORRECT 63
9.2. THE HANA INSTANCE DOES NOT START AFTER HOOK CHANGES 64
9.3. A CLUSTER NODE IS REPORTED AS OFFLINE DURING MAINTENANCE 64

APPENDIX A. COMPONENT OPTIONS 66
A.1. HA/DR PROVIDER OPTIONS FOR HANASR 66
A.2. HA/DR PROVIDER OPTIONS FOR CHKSRV 66
A.3. SAPHANATOPOLOGY RESOURCE PARAMETERS 67
A.4. SAPHANACONTROLLER RESOURCE PARAMETERS 68
A.5. SAPHANAFILESYSTEM RESOURCE PARAMETERS 73

APPENDIX B. USEFUL INFORMATION 74
B.1. EXPLAINING CLUSTER INFORMATION 74

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION

We appreciate your feedback on our documentation. Let us know how we can improve it.

Submitting feedback through Jira (account required)

1. Make sure you are logged in to the Jira website.
2. Click on this link to provide feedback.
3. Enter a descriptive title in the **Summary** field.
4. Enter your suggestion for improvement in the **Description** field. Include links to the relevant parts of the documentation.
5. Click **Create** at the bottom of the dialogue.

CHAPTER 1. INTRODUCTION TO SAP HANA SCALE-UP SYSTEM REPLICATION HA

Configuring the SAP HANA system replication between two identical HANA instances enables a basic resiliency of the database. You can configure these two instances in a Pacemaker cluster for advanced high availability that automatically handles the service recovery in the case of a failure on the primary instance side.

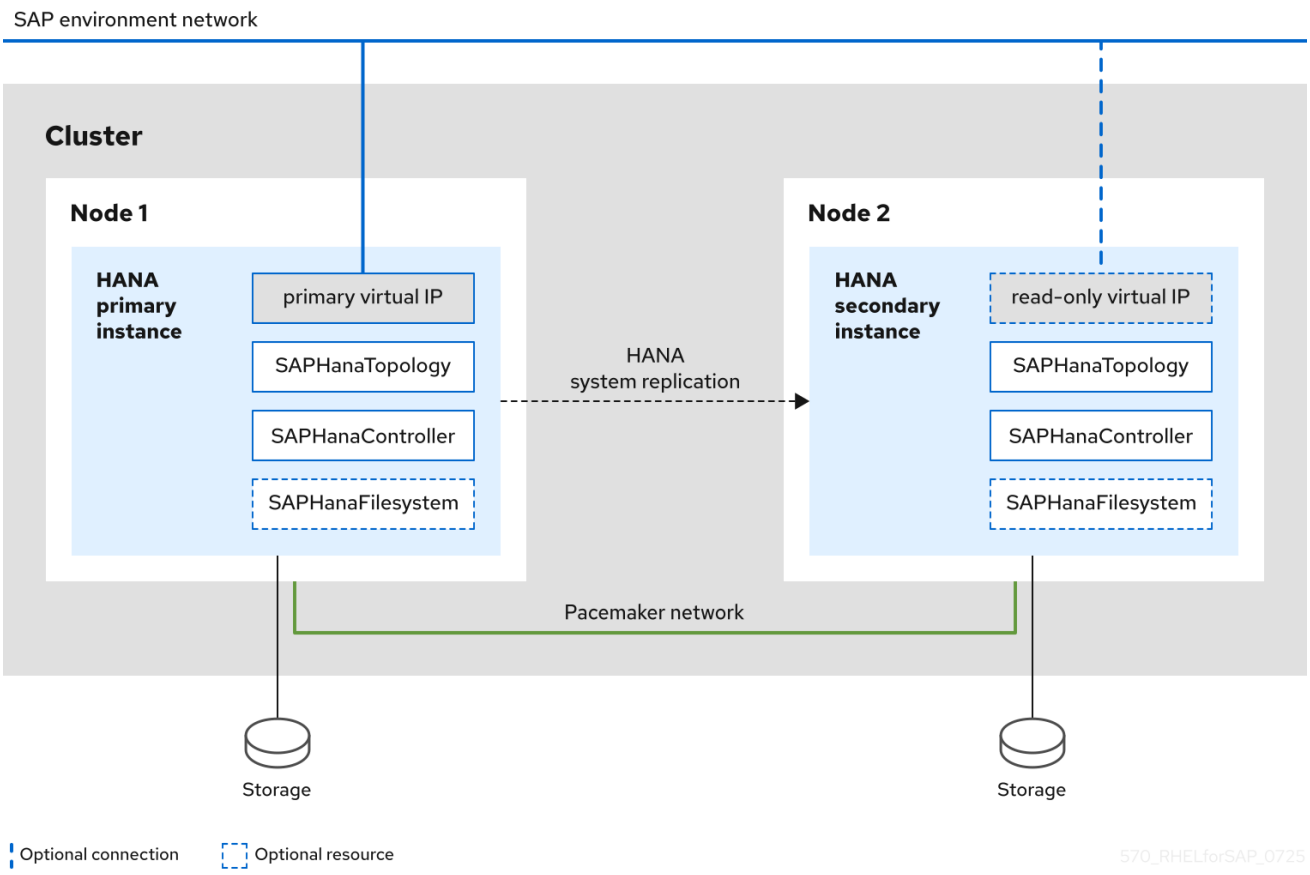
1.1. TERMINOLOGY

- **node**
  One host or system in a HA cluster setup, also called a cluster member.
- **cluster**
  Cluster is the high-availability setup using the Pacemaker cluster manager from the RHEL HA Add-On. It consists of two or more members, or nodes.
- **instance**
  One set of SAP HANA systems that belong to one HANA site. In single-host (scale-up) HANA environments, one HANA site consists of a single HANA instance. In multiple-host (scale-out) HANA configurations, each HANA site consists of two or more HANA instances.
- **primary**
  The primary HANA instance or primary site refers to the instance which is the active HANA instance or site. In single-host setups (scale-up), this is one system. In multiple-host (scale-out) setups, the primary database stretches across multiple systems of one HANA site and the systems have different roles in the HANA environment to distribute load.
- **secondary**
  The secondary HANA instance or secondary site refers to the SAP HANA instance or site which is configured to be synced with the primary HANA instance through the SAP HANA system replication mechanism. This instance preloads the in-memory data of the primary instance and is ready to take over if the primary instance fails.

1.2. PERFORMANCE-OPTIMIZED SAP HANA SCALE-UP HA

Performance-optimized means that there is only a single SAP HANA instance running on each node that has control over most of the resources, such as CPU and RAM, on each node. This means that the SAP HANA instances can run with as much performance as possible.

Figure 1.1: SAP HANA scale-up system replication



With a performance-optimized SAP HANA system replication setup of SAP HANA 2.0 SPS1 or newer you can also configure read access to the secondary system to reduce the load on the primary instance. For more information see the SAP documentation for Active/Active (Read Enabled) configuration.

Additional resources

- Supported Scenarios for Automated SAP HANA Scale-Up System Replication - Performance Optimized
- Configuration guides for the setup on public cloud environments

1.3. CLUSTER RESOURCE AGENTS AND TOOLS FOR SAP HANA HA

The high-availability (HA) cluster configuration for a SAP HANA HA setup works with multiple resource agents that combine their functionality for the expected behavior. The Advanced Next Generation Interface resource agents are identical for scale-up and scale-out environments. In upstream they are also called **SAPHanaSR-angi**.

On RHEL this generation of combined resource agents is provided in the package **sap-hana-ha**.

- SAPHanaTopology
  The **SAPHanaTopology** resource agent gets status information from the SAP HANA environment and saves it to cluster properties. The agent also starts and monitors the local **SAP HostAgent**, which is required for starting, stopping and monitoring the HANA instances. A configuration process in SAP HANA called system replication hook adds replication health information as well to the saved properties. Based on the collected environment data, the resource agent defines a dedicated health score of the cluster node. This scoring is used by the cluster to decide if it must initiate the switch of the system replication from one site to the other.

- SAPHanaController
  The **SAPHanaController** resource agent monitors and manages the SAP HANA environment. In case of a failure of the HANA instance, the resource determines which recovery action it takes and executes the commands for an automatic switch, or it changes the active site of the system replication.

- SAPHanaFilesystem
  The **SAPHanaFilesystem** resource agent checks read and write access of a path on the SAP HANA filesystem and takes action on the primary HANA instance when it finds an issue with the filesystem access. This can considerably accelerate the failover and service recovery.

  Using this resource agent is optional, but we recommend it for triggering a faster failover in an event of the underlying SAP HANA storage failure by fencing the primary node. It only takes action on the primary node and only when the HANA system replication is in sync. This resource only monitors a path on the filesystem, it does not mount or unmount the filesystem. Other mechanisms must be used to manage the filesystem itself. For more details refer to The /etc/fstab file.

- SAPHanaSR-showAttr
  The **SAPHanaSR-showAttr** tool shows cluster attributes for the SAP HANA system replication automation in a preformatted overview. The default output includes the system replication status between the nodes and other related status information, including the HANA topology that shows whether it is a scale-up or scale-out environment. The script retrieves the information from the Cluster Information Base (CIB), where other resource agents or hook scripts store updates during their regular checks or from HANA events, respectively. As a result, the information can contain outdated states until it is updated again. Use HANA tools to get real-time status information from the landscape.

- SAPHanaSR-hookHelper
  The **SAPHanaSR-hookHelper** tool is a helper script that is used by other SAP HANA HA components for specific shared functionality, such as certain fencing features.

- SAPHanaSR-alert-fencing
  The **SAPHanaSR-alert-fencing** script is a cluster alert agent. In scale-out setups with HANA HA, you can configure this alert agent to trigger the cluster to fence all nodes of the same HANA site after one node of that site is fenced due to a failure. This functionality only applies to scale-out configurations and has no effect in a scale-up setup.

NOTE

Verify that the new generation of resource agents is available for your RHEL version. Check Minimum supported package versions for SAP HANA Scale-Up and Scale-Out System Replication HA solutions.

Additional resources

- Appendix A. Component options
- SAP Note 2063657 - SAP HANA System Replication Takeover Decision Guideline
- **ocf_heartbeat_SAPHanaTopology(7)**, **ocf_heartbeat_SAPHanaController(7)** and **ocf_heartbeat_SAPHanaFilesystem(7)** man pages on your system

1.4. SAP HANA HA/DR PROVIDER HOOKS

Current versions of SAP HANA provide an API in the form of hooks that allow the HANA instance to send notifications for certain events, for example the loss or establishment of the system replication. For each event, the HANA instance calls the configured hooks, also called HA/DR providers. Hooks are custom Python scripts which process the events that HANA sends and the scripts can trigger different actions based on the event information.

You must add the HA/DR provider definition to the HANA global configuration to enable the required functionality of triggering additional actions for certain events.

HanaSR for the srConnectionChanged() hook method

The **HanaSR** hook is required for processing the **srConnectionChanged()** hook method. This method is used by the primary HANA instance for a notification of any change in the HANA system replication status. The primary HANA instance calls the **HanaSR** HA/DR provider when a HANA system replication related event occurs. The hook script **HanaSR.py** then parses **srConnectionChanged()** events for the system replication status detail and as a result it updates the **srHook** cluster attribute. This attribute is used by the resource agents to evaluate the landscape health and make decisions. The value of the system replication or sync state defines if the cluster recovers a failed primary instance on the same node or if it triggers a takeover to the secondary. The takeover is only triggered when the system replication is fully in sync, which means the HANA data is consistent between the HANA sites.

IMPORTANT

You must configure the **HanaSR** hook to enable the **srConnectionChanged()** hook method for proper function and full support of the HA cluster setup.

ChkSrv for the srServiceStateChanged() hook method

When the HANA instance detects an issue with a HANA indexserver process it recovers from the problem by stopping and restarting the **hdbindexserver** service automatically through an internal mechanism.

However, especially for very large HANA instances, the **hdbindexserver** service can take a very long time for the stopping phase of this recovery process. Although HANA reports this service degradation not as an error in the HANA landscape, the situation poses a risk to the data consistency if anything else fails in the instance during that time. To improve the unpredictable service recovery time, you can configure the **ChkSrv** hook to stop or kill the entire affected HANA instance instead immediately.

In a setup with automatic failover enabled (**PREFER_SITE_TAKEOVER=true**), the instance stop leads to a takeover if the secondary node is in a healthy state. Otherwise, instance recovery happens locally, but the enforced local instance restart accelerates the process.

The HANA instance calls the **ChkSrv** hook when an event occurs. The hook script **ChkSrv.py** processes the **srServiceStateChanged()** hook method and executes actions based on the results of the filters it applies to event details. This way the **ChkSrv.py** hook script can distinguish a HANA **hdbindexserver** process that is being stopped and restarted by HANA after a failure from the same process being stopped as part of an intended instance shutdown. When the hook script determines that the event is caused by a failure it triggers the configured action.

The **ChkSrv.py** hook script has multiple options to define what happens when an indexserver failure event is detected:

- **ignore**

This action just writes the parsed events and decision information to a dedicated logfile. This is useful for testing and verifying what the hook script would do when activating **stop** or **kill** actions.

- **stop**
  This action executes a graceful **StopSystem** for the instance through the **sapcontrol** command.

- **kill**
  This action executes the **HDB kill-<signal>** command with a default signal 9, which can be configured. The result is the same as when using **stop**, but can be faster.

- **fence**
  This action triggers the cluster to **fence** the node on which the indexserver failed. This option uses the **SAPHanaSR-hookHelper** tool and requires a sudo entry for the tool for the **<sid>adm** user.

NOTE

Any indexserver failure is treated individually by HANA. The same processes are always triggered for every single indexserver issue.

Enabling the **srServiceStateChanged()** hook is optional.

Additional resources

- Appendix A. Component options

1.5. SUPPORT POLICIES FOR SAP HANA HIGH AVAILABILITY

Red Hat supports the following components of the solution:

- Basic operating system configuration for running SAP HANA on RHEL, based on SAP guidelines
- RHEL HA Add-On
- Red Hat HA solutions for SAP HANA system replication

Additional resources

- Support Policies for RHEL High Availability Clusters - Management of SAP HANA in a Cluster
- SAP Note 3108302 - SAP HANA DB: Recommended OS Settings for RHEL 9

CHAPTER 2. PLANNING THE HA CLUSTER SETUP

Plan your setup carefully to ensure that all requirements for the HA cluster configuration for automating the HANA system replication of your HANA landscape are met.

2.1. SUBSCRIPTION AND REPOSITORIES FOR SAP HANA HA

The solutions for SAP HANA in a Pacemaker cluster for High Availability (HA) are provided in dedicated repositories. The RHEL for SAP Solutions subscription is required to access all relevant content. This subscription includes the RHEL HighAvailability repository.

- **High Availability**
  The RHEL HA Add-On's content is stored in a repository named High Availability. The repository ID is represented as **rhel-9-for-<arch>-highavailability-e4s-rpms**.

- **SAP Solutions**
  Name of the repository that contains the SAP HANA specific content. The repository ID is represented as **rhel-9-for-<arch>-sap-solutions-e4s-rpms**.

The **<arch>** denotes the specific hardware architecture:

- **x86_64**

- **ppc64le**

Example list of repositories enabled as part of the RHEL for SAP Solutions subscription:

```
[root]# dnf repolist
Updating Subscription Management repositories.
repo id                                  repo name
rhel-9-for-x86_64-appstream-e4s-rpms       Red Hat Enterprise Linux 9 for x86_64 - AppStream -
Update Services for SAP Solutions (RPMs)
rhel-9-for-x86_64-baseos-e4s-rpms          Red Hat Enterprise Linux 9 for x86_64 - BaseOS - Update
Services for SAP Solutions (RPMs)
rhel-9-for-x86_64-highavailability-e4s-rpms Red Hat Enterprise Linux 9 for x86_64 - High Availability -
Update Services for SAP Solutions (RPMs)
rhel-9-for-x86_64-sap-netweaver-e4s-rpms    Red Hat Enterprise Linux 9 for x86_64 - SAP
NetWeaver - Update Services for SAP Solutions (RPMs)
rhel-9-for-x86_64-sap-solutions-e4s-rpms    Red Hat Enterprise Linux 9 for x86_64 - SAP Solutions -
Update Services for SAP Solutions (RPMs)
```

Additional resources

- Enabling required repositories

- Overview of Red Hat Enterprise Linux for SAP Solutions Subscription

- RHEL for SAP Subscriptions and Repositories

2.2. OPERATING SYSTEM REQUIREMENTS

Check the SAP Note 3108302 - SAP HANA DB: Recommended OS Settings for RHEL 9 for information about minimum Linux kernel versions and other SAP HANA requirements on the operating system level.

Deploy your host operating system as described in Installing RHEL 9 for SAP Solutions .

Root privileges

For the HANA installation and the cluster HA setup the **root** user or a privileged user that can run any sudo commands is required.

Additional resources

- SAP Note 3108316 - Red Hat Enterprise Linux 9.x: Installation and Configuration
- **sudo(8)** man page on your system

2.3. STORAGE REQUIREMENTS

You can find information about the sizing of your SAP HANA database in the SAP HANA Master Guide.

For the setup of a scale-up SAP HANA environment with HANA system replication between two nodes, you must make sure that both systems are equipped with the same size of filesystem space for your HANA database. Plan the required space for local database backup files and log segments in accordance with your specific HANA configuration.

2.4. NETWORK REQUIREMENTS

You can find information about SAP HANA network architecture considerations in the SAP HANA Administration Guide.

For the SAP HANA system replication setup in a HA cluster we recommend that you configure dedicated networks and connections for the cluster communication traffic, which is separate from any HANA network traffic.

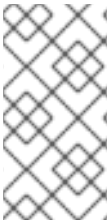
Additional resources

SAP HANA Master Guide

2.5. HA CLUSTER REQUIREMENTS

Fencing

For a supported HA cluster setup using the RHEL HA Add-on you must configure a fencing or STONITH device on each cluster node. Which fencing or STONITH device you can use depends on the platform the cluster is running on. Check the Support Policies for RHEL High Availability Clusters - Fencing/STONITH for recommendations on fencing agents or consult your hardware or cloud provider to find out which fence device is supported on their platform.



NOTE

fence_scsi or **fence_mpath** as fencing/STONITH mechanism requires shared storage between the cluster nodes that is fully managed by the HA cluster. If your SAP environment does not include such a shared disks setup, using these fencing options is not supported.

Additional resources

- High Availability Add-On concepts - Fencing
- Support Policies for RHEL High Availability Clusters - General Requirements for Fencing/STONITH

Quorum

In general, a quorum device is recommended for clusters with an even number of nodes. Two-node clusters specifically have an internal mechanism that handles split-brain situations by itself. For this case a quorum device is optional. Using a quorum device allows the cluster to better determine which node survives in a split-brain situation.

The options for setting up quorum devices vary depending on the platform, infrastructure and configuration.

Additional resources

- High Availability Add-On concepts - Quorum
- Configuring quorum devices
- Exploring Concepts of RHEL High Availability Clusters - Quorum - Red Hat Customer Portal
- Design Guidance for RHEL High Availability Clusters - Considerations with qdevice Quorum Arbitration

2.6. SAP HANA PLANNING

The **sap-hana-ha** package provides the combined resource agents which support setups with SAP HANA 2.0 SPS05 rev 59.04 or newer.

To prepare the HANA setup you can define a list of parameters that you require for the installation and configuration of the planned environment.

Table 2.1. Example SAP HANA configuration parameters

Parameter	Example value
cluster node1 FQDN	**node1.example.com**
cluster node2 FQDN	**node2.example.com**
SID	**RH1**
SAP instance number	**02**
primary HANA site name	**DC1**
secondary HANA site name	**DC2**
HANA **SYSTEMDB** user password	**<HANA_SYSTEM_PASSWORD>**

HANA administrative user	**rh1adm**

Additional resources

- SAP HANA Server Installation and Update Guide
- SAP Note 2057595 - FAQ: SAP HANA High Availability
- SAP Note 3007062 - FAQ: SAP HANA & Third Party Cluster Solutions

CHAPTER 3. INSTALLING SAP HANA FOR A 2-NODE HA CLUSTER SETUP

3.1. INSTALLING A SCALE-UP SAP HANA INSTANCE

Install the HANA instance with the same SID and instance number on all nodes. The setup of the instances must be identical.

Prerequisites

- You have identical systems installed according to the Operating system requirements.
- You have prepared the details for your HANA instances, see SAP HANA planning.
- You have downloaded the SAP HANA installation media from SAP Software Downloads and the media is available on each node.

Procedure

1. Go to the directory which contains the installation media, for example **/sapmedia/hana**:

```
[root]# cd /sapmedia/hana
```

2. Unpack the installation media:

```
[root]# unzip IMDB_SERVER20_*.ZIP
```

3. Go into the path of the unpacked installation media:

```
[root]# cd /sapmedia/hana/DATA_UNITS/HDB_LCM_LINUX_<arch>
```

4. Run the SAP HANA Lifecycle Management tool (HDBLCM) for an interactive installation:

```
[root]# ./hdblcm
```

   In the interactive mode the installer asks you for all the required information, including the System ID (SID), Installation Number (instance), the filesystem location of data and log volumes, and more.

5. Repeat steps 1-4 on all nodes.

Verification

1. Switch to the **<sid>adm** user:

```
[root]# su - rh1adm
```

2. Check the HANA instance runtime information as user **<sid>adm**:

```
rh1adm $ HDB info
USER          PID     PPID  %CPU       VSZ       RSS COMMAND
rh1adm      17897    17895   0.0      8972      5248 -sh
```

```
rh1adm      24461    17897   0.0       7524       3840  \_ /bin/sh /usr/sap/RH1/HDB02/HDB info
rh1adm      24490    24461   0.0      10104       3456      \_ ps fx -U rh1adm -o
user:8,pid:8,ppid:8,pcpu:5,vsz:10,rss:10,args
rh1adm      12924        1   0.0     581456      40072 hdbrsutil  --start --port 34203 --volume 3 --
volumesuffix ...
rh1adm      12284        1   0.0     581376      40000 hdbrsutil  --start --port 34201 --volume 1 --
volumesuffix ...
rh1adm      12193        1   0.0       9028       3028 sapstart
pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_node1
rh1adm      12200    12193   0.0     476372      87652  \_
/usr/sap/RH1/HDB02/node1/trace/hdb.sapRH1_HDB02 -d -nw -f
/usr/sap/RH1/HDB02/node1/daemon.ini pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_node1
rh1adm      12222    12200   9.6   19986956   16126960      \_ hdbnameserver
rh1adm      12787    12200   0.3    1464980     214468      \_ hdbcompileserver
rh1adm      12790    12200  94.0    7669612    6970948      \_ hdbpreprocessor
rh1adm      12828    12200  12.7   19716232   14994012      \_ hdbindexserver -port 34203
rh1adm      12831    12200   1.1    4995088    1465724      \_ hdbxsengine -port 34207
rh1adm      13269    12200   0.4    2660096     461776      \_ hdbwebdispatcher
rh1adm      12016        1   0.0     499404      58336 /usr/sap/RH1/HDB02/exe/sapstartsrv
pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_node1
```

3. Additionally, you can verify the **landscapeHostConfiguration.py** output for status **ok**:

```
rh1adm $ cdpy; python landscapeHostConfiguration.py
| Host  | Host   | Host   | Failover | Remove | Storage   | Storage   | Failover | Failover |
NameServer | NameServer | IndexServer | IndexServer | Host   | Host   | Worker  | Worker  |
|       | Active | Status | Status   | Status | Config    | Actual    | Config   | Actual   | Config     |
Actual     | Config      | Actual      | Config | Actual | Config  | Actual  |
|       |        |        |          |        | Partition | Partition | Group    | Group    | Role       | Role       |
Role       | Role        | Roles  | Roles  | Groups  | Groups  |
| ----- | ------ | ------ | -------- | ------ | --------- | --------- | -------- | -------- | ---------- | ---------- | --
--------- | ----------- | ------ | ------ | ------- | ------- |
| node1 | yes    | ok     |          |        |         1 |         1 | default  | default  | master 1   | master     |
worker      | master      | worker | worker | default | default |

overall host status: ok
```

4. Repeat the steps on all nodes. Note that the HANA profiles contain the individual node name in the format **<SID>_HDB<instance>_<node>**.

Additional resources

SAP HANA Server Installation and Update Guide

3.2. DISABLING SAP HANA INSTANCE AUTOSTART

The cluster controls startup and shutdown of the HANA instance in a HA cluster setup. You must configure the HANA instance profile to not automatically start the instance itself.

Procedure

1. Go to the HANA instance profile directory:

```
[root]# cd /usr/sap/<SID>/SYS/profile
```

2. Edit the instance profile:

```
[root]# vi <SID>_HDB<instance>_<hostname>
```

   Ensure that **Autostart** is set to **0**.

3. Repeat steps 1-2 for each HANA instance that will be managed as part of the HA cluster.

Verification

- Check that **Autostart = 0** is set in the instance profiles of all HANA instances that will be managed by the HA cluster:

```
[root]# grep Autostart /usr/sap/RH1/SYS/profile/*
/usr/sap/RH1/SYS/profile/RH1_HDB02_node1:Autostart = 0
```

Additional resources

SAP HANA Server Installation and Update Guide

CHAPTER 4. CONFIGURING THE SAP HANA SYSTEM REPLICATION

You must configure and test the SAP HANA system replication before you can configure the HANA instance in a cluster. Follow the SAP guidelines for the HANA system replication setup: SAP HANA System Replication: Configuration.

4.1. PREREQUISITES FOR THE SAP HANA SYSTEM REPLICATION SETUP

SAP HANA configuration

SAP HANA must be installed and configured identically on both nodes.

Hostname resolution

Both systems must be able to resolve the FQDN of both systems. To ensure that FQDNs can be resolved even without DNS you can place them into **/etc/hosts**.

Example of **/etc/hosts** entries for the SAP HANA scale-up systems:

```
[root]# cat /etc/hosts
...
192.168.0.11 node1.example.com node1
192.168.0.12 node2.example.com node2
```

NOTE

As documented at hostname | SAP Help Portal , SAP HANA only supports hostnames with lowercase characters.

SAP HANA log_mode

For the system replication to work, you must set the SAP HANA **log_mode** variable to **normal**, which is the default value.

Verify the current **log_mode** as the HANA administrative user **<sid>adm** on both nodes:

```
rh1adm $ hdbsql -u system -p '<HANA_SYSTEM_PASSWORD>' -i ${TINSTANCE} \
"select value from "SYS"."M_INIFILE_CONTENTS" where key='log_mode'"
VALUE "normal"
1 row selected
```

Additional resources

- SAP HANA System Replication: Configuration
- SAP Note 3221437 - System replication is failed due to "Connection refused: Primary has to run in log mode normal for system replication!"

4.2. PERFORMING AN INITIAL HANA INSTANCE BACKUP

You can only enable the HANA system replication when an initial backup of the SAP HANA instance exists on the primary instance for the planned SAP HANA system replication setup.

You can use SAP HANA tools to create the backup and skip the manual procedure. See SAP HANA Administration Guide - SAP HANA Database Backup and Recovery for more information.

Prerequisites

- You have a writable directory to which the backup files are saved for the SAP HANA administrative user **<sid>adm**.
- You have sufficient free space available in the filesystem on which the backup files are stored.

Procedure

1. Optional: Create a dedicated directory for the backup in a suitable path, for example:

```
[root]# mkdir <path>/<SID>-backup
```

   Replace **<path>** with a path on your system, which has enough free space for the initial backup files.

2. Change the owner of the backup path to user **<sid>adm** if the target directory is not already owned or writable by the HANA user, for example:

```
[root]# chown <sid>adm:sapsys <path>/<SID>-backup
```

3. Change to the **<sid>adm** user for the remaining steps:

```
[root]# su - <sid>adm
```

4. Create a backup of the **SYSTEMDB** as the **<sid>adm** user. Specify the path to the files the backups will be stored in. Ensure that the target filesystem has enough free space left, then create the backup:

```
rh1adm $ hdbsql -i ${TINSTANCE} -u system -p '<HANA_SYSTEM_PASSWORD>' -d
SYSTEMDB \
"BACKUP DATA USING FILE ('<path>/${SAPSYSTEMNAME}-backup/bkp-SYS')"
0 rows affected (overall time xx.xxx sec; server time xx.xxx sec)
```

   - **TINSTANCE** and **SAPSYSTEMNAME** are environment variables that are part of the **<sid>adm** user shell environment. **TINSTANCE** is the instance number and **SAPSYSTEMNAME** is the. Both are automatically set to the instance values related to the **<sid>adm** user.
   - Replace **<path>** with the path where the **<sid>adm** user has write access and where there is enough free space left.

5. Create a backup of all tenant databases as the **<sid>adm** user. Specify the path to the files the backups will be stored in. Ensure that the target filesystem has enough free space left. Create the tenant DB backup:

```
rh1adm $ hdbsql -i ${TINSTANCE} -u system -p '<HANA_SYSTEM_PASSWORD>' -d
SYSTEMDB \
```

```
"BACKUP DATA FOR ${SAPSYSTEMNAME} USING FILE ('<path>/${SAPSYSTEMNAME}-backup/bkp-${SAPSYSTEMNAME}')"
0 rows affected (overall time xx.xxx sec; server time xx.xxx sec)
```

Replace **<path>** with the path where the **<sid>adm** user has write access and where there is enough free space left.

Verification

1. List the resulting backup files. Example when using **/hana/log/RH1-backup** as the directory to store the initial backup:

```
rh1adm $ ls -lh /hana/log/RH1-backup/
total 7.2G
-rw-r-----. 1 rh1adm sapsys 156K May  8 08:40 bkp-RH1_databackup_0_1
-rw-r-----. 1 rh1adm sapsys  81M May  8 08:40 bkp-RH1_databackup_2_1
-rw-r-----. 1 rh1adm sapsys 3.6G May  8 08:40 bkp-RH1_databackup_3_1
-rw-r-----. 1 rh1adm sapsys 160K May  8 08:35 bkp-SYS_databackup_0_1
-rw-r-----. 1 rh1adm sapsys 3.6G May  8 08:35 bkp-SYS_databackup_1_1
```

2. Use the HANA command **hdbbackupcheck** to confirm the sanity of each instance backup file you created:

```
rh1adm $ for i in $(ls /hana/log/RH1-backup/*); do hdbbackupcheck $i; done
...
Loaded library 'libhdblivecache'
Backup '/hana/log/RH1-backup/RH1_databackup_0_1' successfully checked.
Loaded library 'libhdbcsaccessor'
...
Loaded library 'libhdblivecache'
Backup '/hana/log/RH1-backup/system_databackup_0_1' successfully checked.
Loaded library 'libhdbcsaccessor'
Loaded library 'libhdblivecache'
Backup '/hana/log/RH1-backup/system_databackup_1_1' successfully checked.
```

Troubleshooting

- The backup fails because the **<sid>adm** user is not able to write to the target directory:

```
* 447: backup could not be completed: [2001003]
createDirectory(path= '/tmp/RH1-backup/', access= rwxrwxr--, recursive= true):
Permission denied (rc= 13, 'Permission denied') SQLSTATE: HY000
```

  Ensure that the **<sid>adm** user can create files inside of the target directory you define in the backup command. Fix the permissions, for example using step 2 of the procedure.

- The backup fails because the target filesystem runs out of space:

```
* 447: backup could not be completed: [2110001]
Generic stream error: $msg$ - , rc=$sysrc$: $sysmsg$.
Failed to process item 0x00007fc5796e0000 - '<root>/.bkp-RH1_databackup_3_1'
((open, mode= W, file_access= rw-r-----, flags=
ASYNC|DIRECT|TRUNCATE|UNALIGNED_SIZE,
size= 4096), factory= (root= '/tmp/RH1-backup/' (root_access= rwxr-x---,
```

```
flags= AUTOCREATE_PATH|DISKFULL_ERROR, usage= DATA_BACKUP, fs= xfs, config=
(async_write_submit_active=on,async_write_submit_blocks=all,async_read_submit=on,num_su
bmit_queues=1,num_completion_queues=1,size_kernel_io_queue=512,max_parallel_io_reques
ts=64,min_submit_batch_size=16,max_submit_batch_size=64))
SQLSTATE: HY000
```

Check the free space of the filesystem on which the target directory is located. Increase the filesystem size or choose a different path with enough free space available for the backup files.

4.3. CONFIGURING THE PRIMARY HANA REPLICATION INSTANCE

Enable the HANA system replication on the system which you plan to be the initial primary instance of your planned system replication setup.

Prerequisites

- You have created an initial backup for the HANA instance on the primary node based on steps described in Performing an initial HANA instance backup .

Procedure

1. Enable the system replication on the HANA instance that becomes the initial primary. Run the command as **<sid>adm** on the first, or primary, node:

```
rh1adm $ hdbnsutil -sr_enable --name=<DC1>
nameserver is active, proceeding ...
successfully enabled system as system replication source site
done.
```

   - Replace **<DC1>** with your primary HANA site name.

Verification

- Check the system replication status as **<sid>adm**, and verify that it shows the current node as **mode: primary**, and that **site id** and **site name** are populated with the primary instance site information:

```
rh1adm $ hdbnsutil -sr_stateConfiguration
System Replication State
~~~~~~~~

mode: primary
site id: 1
site name: DC1
done.
```

4.4. CONFIGURING THE SECONDARY HANA REPLICATION INSTANCE

You must register the secondary HANA instance to the primary instance to complete the setup of the HANA system replication.

Prerequisites

- You have installed SAP HANA on the secondary node using the same **SID** and instance number as the primary instance.
- You have configured **root ssh keys** between the cluster nodes.
- You have opened 2 terminals on the **secondary** node, one for the **root** user and one for the **<sid>adm** user.

Procedure

1. Stop the **secondary** HANA instance and run as the **<sid>adm** user on **node2**:

```
rh1adm $ HDB stop
…
Stop
OK
Waiting for stopped instance using: /usr/sap/<SID>/SYS/exe/hdb/sapcontrol -prot NI_HTTP -
nr <instance> -function WaitforStopped 600 2
…

WaitforStopped
OK
hdbdaemon is stopped.
```

2. Change to the directory where HANA stores the keys of the system replication encryption:

```
[root]# cd /usr/sap/<SID>/SYS/global/security/rsecssfs
```

3. Copy the HANA system PKI file **SSFS_<SID>.KEY** from the primary HANA instance to the secondary instance:

```
[root]# rsync -av node1:$PWD/key/SSFS_<SID>.KEY key/
```

4. Copy the PKI file **SSFS_<SID>.DAT** from the primary HANA instance to the secondary instance:

```
[root]# rsync -av node1:$PWD/data/SSFS_<SID>.DAT data/
```

5. Register the secondary HANA instance to the primary instance, in the **<sid>adm** user terminal:

```
rh1adm $ hdbnsutil -sr_register --remoteHost=node1 \
--remoteInstance=${TINSTANCE} --replicationMode=sync \
--operationMode=logreplay --name=<DC2>
adding site ...
collecting information ...
updating local ini files ...
done.
```

   - Replace **<DC2>** with your secondary HANA site name.
   - Choose the values for **replicationMode** and **operationMode** according to your requirements for the system replication.

- **TINSTANCE** is an environment variable that is set automatically for user **<sid>adm** by reading the HANA instance profile. The variable value is the HANA instance number.

6. Start the secondary HANA instance. Run as **<sid>adm** on **node2**:

```
rh1adm $ HDB start

...
Starting instance using: /usr/sap/<SID>/SYS/exe/hdb/sapcontrol -prot NI_HTTP -nr
<instance> -function StartWait 2700 2

...
```

Verification

1. Check that the system replication is running on the secondary instance and the **mode** matches the value you used for the **replicationMode** parameter in the **hdbnsutil -sr_register** command in step 5. Run as **<sid>adm** on node2:

```
rh1adm $ hdbnsutil -sr_stateConfiguration

System Replication State
~~~~~~~~

mode: sync
site id: 2
site name: DC2
active primary site: 1

primary masters: node1
done.
```

2. Change to the Python script directory of the HANA instance as user **<sid>adm**, on both instances. The easiest way to do this is to use **cdpy**, which is an alias built into the **<sid>adm** user shell that SAP HANA populates during the instance installation:

```
rh1adm $ cdpy
```

In our example this command changes the current directory to **/usr/sap/RH1/HDB02/exe/python_support/**.

3. Show the current status of the established HANA system replication, on both instances.

   a. On the primary instance the system replication status is always displayed with all details:

```
rh1adm $ python systemReplicationStatus.py
|Database |Host     |Port  |Service Name |Volume ID |Site ID |Site Name |Secondary
|Secondary |Secondary |Secondary |Secondary     |Replication |Replication |Replication
|Secondary    |
|         |         |      |             |          |        |          |Host      |Port      |Site ID   |Site Name
|Active Status |Mode        |Status      |Status Details |Fully Synced |
|-------- |-------- |----- |------------ |--------- |------- |--------- |----------|--------- |--------- |-------
-- |------------- |----------- |----------- |-------------- |------------ |
|SYSTEMDB |node1    |30001 |nameserver   |        1 |      1 |DC1       |node2     |    30001 |
```

```
2 |DC2      |YES          |SYNC       |ACTIVE     |              |       True |
|RH1      |node1    |30007 |xsengine     |       2 |      1 |DC1       |node2     |    30007 |       2
|DC2      |YES          |SYNC       |ACTIVE     |              |       True |
|RH1      |node1    |30003 |indexserver  |       3 |      1 |DC1       |node2     |    30003 |       2
|DC2      |YES          |SYNC       |ACTIVE     |              |       True |

status system replication site "2": ACTIVE
overall system replication status: ACTIVE

Local System Replication State
~~~~~~~~~~

mode: PRIMARY
site id: 1
site name: DC1
```

- **ACTIVE** means that the HANA system replication is in a healthy state and fully synced.
- **SYNCING** is displayed while the system replication is being updated on the secondary site, for example after a takeover of the secondary instance to become the new primary.
- **INITIALIZING** is shown after the system replication has first been enabled or a full sync has been triggered.

b. On the **secondary** instance, the output of **systemReplicationStatus.py** is less detailed:

```
rh1adm $ python systemReplicationStatus.py
this system is either not running or not primary system replication site

Local System Replication State
~~~~~~~~~~

mode: SYNC
site id: 2
site name: DC2
active primary site: 1
primary masters: node1
```

Additional resources

- SAP Note 2369981 - Required configuration steps for authentication with HANA System Replication
- Replication Modes for SAP HANA System Replication
- Operation Modes for SAP HANA System Replication

4.5. TESTING THE HANA SYSTEM REPLICATION

We recommend that you test the HANA system replication thoroughly before you proceed with the cluster setup. The verification of the correct system replication behavior can help to prevent unexpected results when the HA cluster manages the system replication afterwards.

Use timeout values in the cluster resource configuration that cover the measured times of the different tests to ensure that cluster resource operations do not time out prematurely.

You can also test different parameter values in the HANA configuration to optimize the performance by measuring the time that certain activities take when performed manually outside of cluster control.

Perform the tests with realistic data loads and sizes.

Full replication

- How long does the synchronization take after the newly registered secondary is started until it is in sync with the primary?
- Are there parameters which can improve the synchronization time?

Lost connection

- How long does it take after the connection was lost between the primary and the secondary instance, until they are in sync again?
- Are there parameters which can improve the reconnection and sync times?

Takeover

- How long does the secondary instance take to be fully available after a takeover from the primary?
- What is the time difference between a normal takeover and a "takeover with handshake"?
- Are there parameters which can improve the takeover time?

Data consistency

- Is the data you create available and correct after you perform a takeover?

Client reconnect

- Can the client reconnect to the new primary instance after a takeover?
- How long does it take for the client to access the new primary after a takeover?

Primary becomes secondary

- How long does it take a former primary until it is in sync again with the new primary, after it is registered as a new secondary?
- If configured, how long does it take until a client can access the newly registered secondary for read operations?

Additional resources

Refer to section “9. Testing”, in How To Perform System Replication for SAP HANA , for more information.

CHAPTER 5. CONFIGURING THE PACEMAKER CLUSTER

5.1. DEPLOYING THE BASIC CLUSTER CONFIGURATION

The following basic cluster setup covers the minimum steps to get started with the Pacemaker cluster setup for managing SAP HANA system replication.

For more information on settings and options for complex configurations, refer to the documentation for RHEL HA Add-On, for example, Create a high availability cluster with multiple links .

Prerequisites

- You have set up the HANA system replication environment and verified that it is working correctly.
- You have configured the RHEL High Availability repository on all systems that are going to be nodes of this cluster.
- You have verified fencing and quorum requirements according to your planned environment. For more details see HA cluster requirements.

Procedure

1. Install the Red Hat High Availability Add-On software packages from the High Availability repository. Choose which fence agents you want to install and execute the installation on all cluster nodes.

   a. Either install the cluster packages and all fence agents:

   ```
   [root]# dnf install pcs pacemaker fence-agents-all
   ```

   b. Or install the cluster packages and only a specific fence agent, depending on your environment:

   ```
   [root]# dnf install pcs pacemaker fence-agents-<model>
   ```

2. Start and enable the **pcsd** service on all cluster nodes:

   ```
   [root]# systemctl enable --now pcsd.service
   ```

3. Optional: If you are running the **firewalld** service, enable the ports that are required by the Red Hat High Availability Add-On. Run this on all cluster nodes:

   ```
   [root]# firewall-cmd --add-service=high-availability
   [root]# firewall-cmd --runtime-to-permanent
   ```

4. Set a password for the user **hacluster**. Repeat the command on each node using the same password:

   ```
   [root]# passwd hacluster
   ```

5. Authenticate the user **hacluster** for each node in the cluster. Run this on the first node:

```
[root]# pcs host auth <node1> <node2>
Username: hacluster
Password:
<node1>: Authorized
<node2>: Authorized
```

- Enter the node names with or without FQDN, as defined in the **/etc/hosts** file.
- Enter the **hacluster** user password in the prompt.

6. Create the cluster with a name and provide the names of the cluster members, for example **node1** and **node2** with fully qualified host names. This propagates the cluster configuration on both nodes and starts the cluster. Run this command on the first node:

```
[root]# pcs cluster setup <cluster_name> --start <node1> <node2>
No addresses specified for host 'node1', using 'node1'
No addresses specified for host 'node2', using 'node2'
Destroying cluster on hosts: 'node1', 'node2'...
node2: Successfully destroyed cluster
node1: Successfully destroyed cluster
Requesting remove 'pcsd settings' from 'node1', 'node2'
node1: successful removal of the file 'pcsd settings'
node2: successful removal of the file 'pcsd settings'
Sending 'corosync authkey', 'pacemaker authkey' to 'node1', 'node2'
node1: successful distribution of the file 'corosync authkey'
node1: successful distribution of the file 'pacemaker authkey'
node2: successful distribution of the file 'corosync authkey'
node2: successful distribution of the file 'pacemaker authkey'
Sending 'corosync.conf' to 'node1', 'node2'
node1: successful distribution of the file 'corosync.conf'
node2: successful distribution of the file 'corosync.conf'
Cluster has been successfully set up.
Starting cluster on hosts: 'node1', 'node2'...
```

7. Enable the cluster to be started automatically on system start, which enables the **corosync** and **pacemaker** services. Skip this step if you prefer to manually control the start of the cluster after a node restarts. Run on one node:

```
[root]# pcs cluster enable --all
node1: Cluster Enabled
node2: Cluster Enabled
```

Verification

- Check the cluster status. Verify that the cluster daemon services are in the desired state:

```
[root]# pcs status --full
Cluster name: node1-node2-cluster

WARNINGS:
No stonith devices and stonith-enabled is not false

Cluster Status:
 Cluster Summary:
   * Stack: corosync (Pacemaker is running)
```

```
  * Current DC: node1 (version ) - partition with quorum * Last updated: * on node1 * Last
change: ** by hacluster via hacluster on node1
  * 2 nodes configured
  * 0 resource instances configured

...

PCSD Status:
  node1: Online
  node2: Online

Daemon Status:
  corosync: active/enabled
  pacemaker: active/enabled
  pcsd: active/enabled
```

Next steps

- Configure a fencing method to enable the STONITH mechanism. See Configuring fencing in a Red Hat High Availability cluster.
- Test the fencing setup before you proceed with further configuration of the cluster. For more information, see How to test fence devices and fencing configuration in a Red Hat High Availability cluster?

Additional resources

- Creating a Red Hat High-Availability cluster with Pacemaker
- Support Policies for RHEL High Availability Clusters

5.2. CONFIGURING GENERAL CLUSTER PROPERTIES

You must adjust cluster resource defaults to avoid unnecessary failovers of the resources.

Procedure

- Update cluster resource defaults to avoid unnecessary failovers of the resources. Run the command on one cluster node to apply the change to the cluster configuration:

```
[root]# pcs resource defaults update \
resource-stickiness=1000 \
migration-threshold=5000
```

  - **resource-stickiness=1000** encourages the resource to stay running where it is. This prevents the cluster from moving the resources based on light internal health indicators.
  - **migration-threshold=5000** enables the resource to be restarted on a node after 5000 failures. After exceeding this limit, the resource is blocked on the node until the failure has been cleared. This allows resource recovery after a few failures until an administrator can investigate the cause of the repeated failure and reset the counter.

Verification

- Check that the resource defaults are set:

```
[root]# pcs resource defaults
Meta Attrs: build-resource-defaults
  migration-threshold=5000
  resource-stickiness=1000
```

5.3. CONFIGURING THE SYSTEMD-BASED SAP STARTUP FRAMEWORK

Systemd integration is the default behavior of SAP HANA installations on RHEL 9 for SAP HANA 2.0 SPS07 revision 70 and newer. In HA environments you must apply additional modifications to integrate the different systemd services that are involved in the cluster setup.

Configure the pacemaker systemd service to manage the HANA instance systemd service in the correct order.

Prerequisites

- You have installed the HANA instance with systemd integration and have checked on all nodes, for example:

```
[root]# systemctl list-units --all SAP*
  UNIT              LOAD      ACTIVE   SUB     DESCRIPTION
  SAPRH1_02.service loaded    active   running SAP Instance SAPRH1_02
  SAP.slice         loaded    active   active  SAP Slice
...
```

Procedure

1. Create the directory **/etc/systemd/system/pacemaker.service.d/** for the pacemaker service drop-in file:

```
[root]# mkdir /etc/systemd/system/pacemaker.service.d/
```

2. Create the **systemd** drop-in file for the pacemaker service with the following content:

```
[root]# cat << EOF > /etc/systemd/system/pacemaker.service.d/00-pacemaker.conf
[Unit]
Description=Pacemaker needs the SAP HANA instance service
Wants=SAP<SID>_<instance>.service
After=SAP<SID>_<instance>.service
EOF
```

   - Replace **<SID>** with your HANA SID.
   - Replace **<instance>** with your HANA instance number.

3. Reload the **systemctl** daemon to activate the drop-in file:

```
[root]# systemctl daemon-reload
```

4. Repeat steps 1-3 on the other cluster nodes.

Verification

1. Check the systemd service of your HANA instance and verify that it is **loaded**:

```
[root]# systemctl status SAPRH1_02.service
● SAPRH1_02.service - SAP Instance SAPRH1_02
    Loaded: loaded (/etc/systemd/system/SAPRH1_02.service; disabled; preset: disabled)
    Active: active (running) since xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
  Main PID: 5825 (sapstartsrv)
     Tasks: 841
    Memory: 88.6G
       CPU: 4h 50min 2.033s
    CGroup: /SAP.slice/SAPRH1_02.service
            ├─ 5825 /usr/sap/RH1/HDB02/exe/sapstartsrv
pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_node1
            ├─ 5986 sapstart pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_node1
            ├─ 5993 /usr/sap/RH1/HDB02/node1/trace/hdb.sapRH1_HDB02 -d -nw -f
/usr/sap/RH1/HDB02/node1/daemon.ini pf=/usr/sap/RH1/SYS/profile/RH1_HDB02_node1
...
```

2. Verify that the SAP HANA instance service is known to the pacemaker service now:

```
[root]# systemctl show pacemaker.service | grep SAP
Wants=SAPRH1_02.service resource-agents-deps.target dbus-broker.service
After=... SAPRH1_02.service rsyslog.service
```

Make sure that the **SAP<SID>_<instance>.service** is listed in the **After=** and **Wants=** lists.

Additional resources

- The Systemd-Based SAP Startup Framework
- Changes required for managing systemd enabled SAP HANA or "anydb" instances managed by the RHEL HA Add-On
- SAP Note 3139184 - Linux: systemd integration for sapstartsrv and SAP Host Agent
- SAP Note 3189534 - Linux: systemd integration for sapstartsrv and SAP HANA
- **systemctl(1)** man page on your system

5.4. INSTALLING THE SAP HANA HA COMPONENTS

The **sap-hana-ha** RPM package in the **Red Hat Enterprise Linux 9 for <arch> - SAP Solutions (RPMs)** repository provides resource agents and other SAP HANA specific components for setting up a HA cluster for managing HANA system replication setup.

IMPORTANT

The package **sap-hana-ha** is only available since RHEL 9.4. If you configure a cluster for HANA HA on an older RHEL 9 release, follow the instructions in Automating SAP HANA Scale-Up System Replication using the RHEL HA Add-On instead.

Procedure

- Install the **sap-hana-ha** package on all cluster nodes:

```
[root]# dnf install sap-hana-ha
```

Verification

- Check on all nodes that the package is installed:

```
[root]# rpm -q sap-hana-ha
sap-hana-ha-<version>.<release>.noarch
```

Additional resources

- Support Policies for RHEL High Availability Clusters - Management of SAP HANA in a Cluster

5.5. CONFIGURING THE HANASR HA/DR PROVIDER FOR THE SRCONNECTIONCHANGED() HOOK METHOD

When you configure the HANA instance in a HA cluster setup with SAP HANA 2.0 SPS0 or later, you must enable and test the SAP HANA **srConnectionChanged()** hook method before proceeding with the cluster setup.

Prerequisites

- You have installed the **sap-hana-ha** package.
- Your HANA instance is not yet managed by the cluster. Otherwise, use the maintenance procedure Updating the SAP HANA instances to make sure that the cluster does not interfere during the configuration of the hook scripts.

Procedure

1. Stop the HANA instances on all nodes as the **<sid>adm** user:

```
rh1adm $ HDB stop
```

2. Verify as **<sid>adm** on all nodes that the HANA instances are stopped completely:

```
rh1adm $ sapcontrol -nr ${TINSTANCE} -function GetProcessList | column -s ',' -t
...
name          description   dispstatus   textstatus   starttime   elapsedtime   pid
hdbdaemon     HDB Daemon    GRAY         Stopped                               5381
```

3. Change to the HANA configuration directory, as the **<sid>adm** user, using the command alias **cdcoc**, which is built into the**<sid>adm** user shell. This automatically changes into the **/hana/shared/<SID>/global/hdb/custom/config/** path:

```
rh1adm $ cdcoc
```

4. Update the **global.ini** file of the SAP HANA instance to configure the **HanaSR** hook. Edit the configuration file on all HANA instance nodes and add the following configuration:

```
[ha_dr_provider_hanasr]
provider = HanaSR
path = /usr/share/sap-hana-ha/
execution_order = 1

[trace]
ha_dr_hanasr = info
```

   Set **execution_order** to **1** to ensure that the **HanaSR** hook is always executed with the highest priority.

5. Optional: When you also want to configure the optional **ChkSrv** hook for taking action on an **hdbindexserver** failure, you can add the changes to the **global.ini** at the same time, see step 1 in Configuring the ChkSrv HA/DR provider for the srServiceStateChanged() hook method:

```
[ha_dr_provider_hanasr]
provider = HanaSR
path = /usr/share/sap-hana-ha/
execution_order = 1

[ha_dr_provider_chksrv]
provider = ChkSrv
path = /usr/share/sap-hana-ha/
execution_order = 2
action_on_lost = stop

[trace]
ha_dr_hanasr = info
ha_dr_chksrv = info
```

6. Create the file **/etc/sudoers.d/20-saphana**, as the **root** user, on each cluster node with the following content. These command privileges allow the **<sid>adm** user to update certain cluster node attributes as part of the HanaSR hook execution:

```
[root]# visudo -f /etc/sudoers.d/20-saphana
<sid>adm ALL=(ALL) NOPASSWD: /usr/sbin/crm_attribute -n hana_*
Defaults:<sid>adm !requiretty
```

   - For further information on why the **Defaults** setting is needed, refer to The srHook attribute is set to SFAIL in a Pacemaker cluster managing SAP HANA system replication, even though replication is in a healthy state.

7. Start the HANA instances on both cluster nodes manually without starting the HA cluster. Run as **<sid>adm**:

```
rh1adm $ HDB start
```

Verification

1. Change to the SAP HANA directory, as the **<sid>adm** user, where trace log files are stored. Use the command alias **cdtrace**, which is built into the **<sid>adm** user shell:

```
rh1adm $ cdtrace
```

2. Check the HANA nameserver service logs for the HA/DR provider loading message:

   a. If only the **HanaSR** provider is configured:

```
rh1adm $ grep -he "loading HA/DR Provider.*" nameserver_*
loading HA/DR Provider 'HanaSR' from /usr/share/sap-hana-ha/
```

   b. If the optional **ChkSrv** provider is also implemented:

```
rh1adm $ grep -he "loading HA/DR Provider.*" nameserver_*
loading HA/DR Provider 'ChkSrv' from /usr/share/sap-hana-ha/
loading HA/DR Provider 'HanaSR' from /usr/share/sap-hana-ha/
```

3. Verify as user **root** in the system secure log that the **sudo** command executed successfully. If the sudoers file is not correct, an error is logged when the **sudo** command is executed:

```
[root]# grep -e 'sudo.*crm_attribute.*' /var/log/secure
 sudo[4267]:  rh1adm : PWD=/hana/shared/RH1/HDB02/node1 ; USER=root ;
COMMAND=/usr/sbin/crm_attribute -n hana_rh1_site_srHook_DC2 -v SFAIL -t crm_config -s
SAPHanaSR
 sudo[4319]:  rh1adm : PWD=/hana/shared/RH1/HDB02/node1 ; USER=root ;
COMMAND=/usr/sbin/crm_attribute -n hana_rh1_site_srHook_DC2 -v SOK -t crm_config -s
SAPHanaSR
```

   After the HANA instance starts on both nodes, you can usually see several **srHook** attribute updates. At first it is setting **SFAIL**, because immediately after the primary starts, it is not yet in sync with the secondary, which is still synchronizing at this time.

   The last update to **SOK** is triggered by the HANA event after the system replication status finally changes to fully in sync.

4. Repeat the verification steps 1-2 on the second instance, if not already done at the same time. The **sudo** log messages of step 3 are only visible on the primary instance node where the system replication events are sent.

5. Check the cluster attributes on any node and verify that the value for the **hana_<sid>_site_srHook_<DC2>** attribute is updated as expected:

```
[root]# cibadmin --query | grep -e 'SAPHanaSR.*srHook'
        <nvpair id="SAPHanaSR-hana_rh1_site_srHook_DC2"
name="hana_rh1_site_srHook_DC2" value="SOK"/>
```

   - **SOK** is set when the HANA system replication is in **ACTIVE** state, which means established and fully in sync.
   - **SFAIL** is set when the system replication is in any other state.

Troubleshooting

- Refer to The HANA instance does not start after hook changes

Additional resources

- Appendix A. Component options - HA/DR provider options for HanaSR
- Install and Configure a HA/DR Provider Script
- How can the srConnectionChanged() hook be used to improve the detection of situations where a takeover is required, in a Red Hat Pacemaker cluster managing HANA Scale-up or Scale-out System Replication?
- **sudo(8)** and **visudo(8)** man pages on your system

5.6. CONFIGURING THE CHKSRV HA/DR PROVIDER FOR THE SRSERVICESTATECHANGED() HOOK METHOD

You can configure the hook **ChkSrv** if you want the HANA instance to be stopped or killed for faster recovery after an indexserver process has failed. This configuration is optional.

Prerequisites

- You have installed the **sap-hana-ha** package.
- You have configured the **HanaSR** HA/DR provider. For more information see Configuring the HanaSR HA/DR provider for the srConnectionChanged() hook method.

Procedure

1. Change to the HANA configuration directory as the **<sid>adm** user. Use the command alias **cdcoc**, which is built into the **<sid>adm** user shell. This automatically changes into the **/hana/shared/<SID>/global/hdb/custom/config/** path:

```
rh1adm $ cdcoc
```

2. Update the **global.ini** file of the HANA instance to configure the hook script. Edit the configuration file on all HANA instance systems and add the following content in addition to the **HanaSR** provider definition:

```
[ha_dr_provider_chksrv]
provider = ChkSrv
path = /usr/share/sap-hana-ha/
execution_order = 2
action_on_lost = stop

[trace]
ha_dr_hanasr = info
ha_dr_chksrv = info
```

3. Optional: If you want to use the **fence** option for **action_on_lost**, you must add the **SAPHanaSR-hookHelper** to the sudo configuration of the **<sid>adm** user:

```
[root]# visudo -f /etc/sudoers.d/20-saphana
…
<sid>adm ALL=(ALL) NOPASSWD: /usr/bin/SAPHanaSR-hookHelper
```

4. Optional: Activate the **ChkSrv** provider while HANA is running by reloading the HA/DR providers. Skip this step when configuring the hook script while the instance is down, the HA/DR provider is loaded automatically at the next instance start.

```
rh1adm $ hdbnsutil -reloadHADRProviders
```

Verification

1. Change to the SAP HANA directory, as the **<sid>adm** user, where trace log files are stored. Use the command alias **cdtrace**, which is built into the**<sid>adm** user shell:

```
rh1adm $ cdtrace
```

2. Check that the changes are loaded:

```
rh1adm $ grep -e "loading HA/DR Provider.*ChkSrv.*" nameserver_*
loading HA/DR Provider 'ChkSrv' from /usr/share/sap-hana-ha/
```

3. Check that the dedicated trace log file is created and the provider loaded with the correct configuration parameters:

```
rh1adm $ cat nameserver_chksrv.trc
init called
ChkSrv.init() version 1.001.1, parameter info: action_on_lost=stop stop_timeout=20
kill_signal=9
```

Troubleshooting

- Refer to The HANA instance does not start after hook changes

Additional resources

- Appendix - A.2. HA/DR provider options for ChkSrv
- Implementing a HA/DR Provider

5.7. CREATING THE HANA CLUSTER RESOURCES

You must configure both the **SAPHanaTopology** and **SAPHanaController** resources so that the cluster can collect the status of the HANA landscape, monitor the instance health and take action to manage the instance when required.

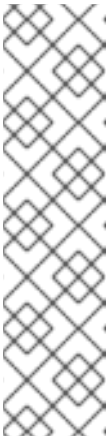
The **SAPHanaFilesystem** resource is optional, you can add it for improving the time to action in case the filesystem of the primary instance becomes unavailable.

Procedure

1. Create the **SAPHanaTopology** resource as a clone resource, which means it runs on all cluster nodes at the same time:

```
[root]# pcs resource create rsc_SAPHanaTop_<SID>_HDB<instance> \
ocf:heartbeat:SAPHanaTopology \
SID=<SID> \
InstanceNumber=<instance> \
op start timeout=600 \
op stop timeout=300 \
op monitor interval=30 timeout=300 \
clone cln_SAPHanaTop_<SID>_HDB<instance> \
meta clone-max=2 clone-node-max=1 interleave=true --future
```

- Replace **<SID>** with your HANA SID.
- Replace **<instance>** with your HANA instance number.



NOTE

Since RHEL 9.3, a deprecation warning is displayed when not using the **meta** keyword in the clone command and the attributes are automatically assigned to the base resource.

In the future, the **meta** keyword for clone attributes will be required for assigning attributes to the clone resource. Until then, add the **--future** parameter to already apply this behavior now.

See also New pcs parsing requires meta keyword when specifying clone meta attributes.

2. Create the **SAPHanaController** resource as a promotable clone resource. This means it runs on all cluster nodes at the same time, but on one node it functions as the active, or primary, instance:

```
[root]# pcs resource create rsc_SAPHanaCon_<SID>_HDB<instance> \
ocf:heartbeat:SAPHanaController \
SID=<SID> \
InstanceNumber=<instance> \
PREFER_SITE_TAKEOVER=true \
DUPLICATE_PRIMARY_TIMEOUT=7200 \
AUTOMATED_REGISTER=false \
op stop timeout=3600 \
op monitor interval=59 role=Promoted timeout=700 \
op monitor interval=61 role=Unpromoted timeout=700 \
meta priority=100 \
promotable cln_SAPHanaCon_<SID>_HDB<instance> \
meta clone-max=2 clone-node-max=1 interleave=true --future
```

We recommend creating the resource with **AUTOMATED_REGISTER=false** and then verify the correct behavior and data consistency through tests to complete the setup. For more information, see Testing the setup. You can enable this already at creation time by setting the parameter to true.

See SAPHanaController resource parameters for more details.

3. Optional: If you want the cluster to fence the node when the **SAPHanaController** resource fails, then update the resource with the **ON_FAIL_ACTION** parameter and set it to **fence**:

```
[root]# pcs resource update rsc_SAPHanaCon_<SID>_HDB<instance>
ON_FAIL_ACTION=fence
```

4. You must start **SAPHanaTopology** before **SAPHanaController**, because it collects HANA landscape information, which the **SAPHanaController** resource requires to start correctly. Create the cluster constraint that enforces the correct start order of the two resources:

```
[root]# pcs constraint order cln_SAPHanaTop_<SID>_HDB<instance> \
then cln_SAPHanaCon_<SID>_HDB<instance> symmetrical=false
```

Setting **symmetrical=false** indicates that the constraint only influences the start order of the resources, but it does not apply to the stop order.

5. Optional: Create the **SAPHanaFilesystem** resource as a clone resource:

```
[root]# pcs resource create rsc_SAPHanaFil_<SID>_HDB<instance> \
ocf:heartbeat:SAPHanaFilesystem \
SID=<SID> \
InstanceNumber=<instance> \
ON_FAIL_ACTION="fence" \
op start interval=0 timeout=10 \
op stop interval=0 timeout=20 \
op monitor interval=120 timeout=120 \
clone cln_SAPHanaFil_<SID>_HDB<instance> \
meta clone-node-max=1 interleave=true --future
```

Instead of setting **ON_FAIL_ACTION=fence** you can set it to **ignore**. This can be useful for testing the functionality first. The resource writes information to the system logs, which can be examined for evaluating if the resource would take the wanted action when activated to use the **fence** action.

Verification

1. Review the **SAPHanaTopology** resource clone. Example resource configuration:

```
[root]# pcs resource config cln_SAPHanaTop_RH1_HDB02
Clone: cln_SAPHanaTop_RH1_HDB02
  Meta Attributes: cln_SAPHanaTop_RH1_HDB02-meta_attributes
    clone-max=2
    clone-node-max=1
    interleave=true
  Resource: rsc_SAPHanaTop_RH1_HDB02 (class=ocf provider=heartbeat
type=SAPHanaTopology)
    Attributes: rsc_SAPHanaTop_RH1_HDB02-instance_attributes
      InstanceNumber=02
      SID=RH1
    Operations:
      methods: rsc_SAPHanaTop_RH1_HDB02-methods-interval-0s
        interval=0s timeout=5
      monitor: rsc_SAPHanaTop_RH1_HDB02-monitor-interval-30
        interval=30 timeout=300
```

```
      reload: rsc_SAPHanaTop_RH1_HDB02-reload-interval-0s
        interval=0s timeout=5
      start: rsc_SAPHanaTop_RH1_HDB02-start-interval-0s
        interval=0s timeout=600
      stop: rsc_SAPHanaTop_RH1_HDB02-stop-interval-0s
        interval=0s timeout=300
```

2. **Review the SAPHanaController resource clone. Example resource configuration:**

```
[root]# pcs resource config cln_SAPHanaCon_RH1_HDB02
Clone: cln_SAPHanaCon_RH1_HDB02
  Meta Attributes: cln_SAPHanaCon_RH1_HDB02-meta_attributes
    clone-max=2
    clone-node-max=1
    interleave=true
    promotable=true
  Resource: rsc_SAPHanaCon_RH1_HDB02 (class=ocf provider=heartbeat
type=SAPHanaController)
    Attributes: rsc_SAPHanaCon_RH1_HDB02-instance_attributes
      AUTOMATED_REGISTER=false
      DUPLICATE_PRIMARY_TIMEOUT=7200
      InstanceNumber=02
      PREFER_SITE_TAKEOVER=true
      SID=RH1
    Meta Attributes: rsc_SAPHanaCon_RH1_HDB02-meta_attributes
      priority=100
    Operations:
      demote: rsc_SAPHanaCon_RH1_HDB02-demote-interval-0s
        interval=0s timeout=320
      methods: rsc_SAPHanaCon_RH1_HDB02-methods-interval-0s
        interval=0s timeout=5
      monitor: rsc_SAPHanaCon_RH1_HDB02-monitor-interval-59
        interval=59 timeout=700 role=Promoted
      monitor: rsc_SAPHanaCon_RH1_HDB02-monitor-interval-61
        interval=61 timeout=700 role=Unpromoted
      promote: rsc_SAPHanaCon_RH1_HDB02-promote-interval-0s
        interval=0s timeout=900
      reload: rsc_SAPHanaCon_RH1_HDB02-reload-interval-0s
        interval=0s timeout=5
      start: rsc_SAPHanaCon_RH1_HDB02-start-interval-0s
        interval=0s timeout=3600
      stop: rsc_SAPHanaCon_RH1_HDB02-stop-interval-0s
        interval=0s timeout=3600
```

3. **Check that the start order constraint is in place:**

```
[root]# pcs constraint order
Order Constraints:
  start resource 'cln_SAPHanaTop_RH1_HDB02' then start resource
'cln_SAPHanaCon_RH1_HDB02'
    symmetrical=0
```

4. **Optional: Review the SAPHanaFilesystem resource clone if you created it. Example resource configuration:**

```
[root]# pcs resource config cln_SAPHanaFil_RH1_HDB02
Clone: cln_SAPHanaFil_RH1_HDB02
  Meta Attributes: cln_SAPHanaFil_RH1_HDB02-meta_attributes
    clone-node-max=1
    interleave=true
  Resource: rsc_SAPHanaFil_RH1_HDB02 (class=ocf provider=heartbeat
type=SAPHanaFilesystem)
    Attributes: rsc_SAPHanaFil_RH1_HDB02-instance_attributes
      InstanceNumber=02
      ON_FAIL_ACTION=fence
      SID=RH1
    Operations:
      methods: rsc_SAPHanaFil_RH1_HDB02-methods-interval-0s
        interval=0s timeout=5
      monitor: rsc_SAPHanaFil_RH1_HDB02-monitor-interval-120
        interval=120 timeout=120
      reload: rsc_SAPHanaFil_RH1_HDB02-reload-interval-0s
        interval=0s timeout=5
      start: rsc_SAPHanaFil_RH1_HDB02-start-interval-0
        interval=0 timeout=10
      stop: rsc_SAPHanaFil_RH1_HDB02-stop-interval-0
        interval=0 timeout=20
```

5. **Check the cluster status. Use --full to include node attributes, which are updated by the HANA resources:**

```
[root]# pcs status --full
...

Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]:
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node1
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node2
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02]
(promotable):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Promoted
node1
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted
node2
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02]:
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started node1
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started node2

Node Attributes:
  * Node: node1 (1):
    * hana_rh1_clone_state              : PROMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC1
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.078.00
    * hana_rh1_vhost                    : node1
    * master-rsc_SAPHanaCon_RH1_HDB02   : 150
  * Node: node2 (2):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : master1:master:worker:master
```

```
    * hana_rh1_site                    : DC2
    * hana_rh1_srah                    : -
    * hana_rh1_version                 : 2.00.078.00
    * hana_rh1_vhost                   : node2
    * master-rsc_SAPHanaCon_RH1_HDB02   : 100

...
```

NOTE

The timeouts shown for the resource operations are only recommended defaults and can be adjusted depending on your SAP HANA environment. For example, large SAP HANA databases can take longer to start up and therefore you might have to increase the start timeout.

WARNING

Setting **AUTOMATED_REGISTER** to **true** can potentially increase the risk of data loss or corruption. If the HA cluster triggers a takeover when the data on the secondary HANA instance is not fully in sync, the automatic registration of the old primary HANA instance as the new secondary HANA instance results a data loss on this instance and any data that was not synced before the takeover occurred is lost as well.

For more information, see the article on the SAP Technology Blog for Members: Be Prepared for Using Pacemaker Cluster for SAP HANA – Part 2: Failure of Both Nodes.

Additional resources

- Appendix A. Component options
- You can also view all available resource options using **pcs resource describe <resource_agent_name>**.
- **pcs(8)**, **ocf_heartbeat_SAPHanaTopology(7)**, **ocf_heartbeat_SAPHanaController(7)** and **ocf_heartbeat_SAPHanaFilesystem(7)** man pages on your system

5.8. CREATING THE VIRTUAL IP RESOURCE

You must configure a virtual IP (VIP) resource for SAP clients to access the primary HANA instance independently from the cluster node it is currently running on. Configure the VIP resource to automatically move to the node where the primary instance is running.

The resource agent needed for the VIP resource depends on the platform used. We are using the IPaddr2 resource agent to demonstrate the setup.

Prerequisites

- You have reserved a virtual IP for the service.

Procedure

1. Use the appropriate resource agent for managing the virtual IP address based on the platform on which the HA cluster is running. Adjust the parameters according to the resource agent you are using. Create the cluster resource for the primary virtual IP, for example using the **IPaddr2** agent:

```
[root]# pcs resource create rsc_vip_<SID>_HDB<instance>_primary \
ocf:heartbeat:IPaddr2 ip=<address> cidr_netmask=<netmask> nic=<device>
```

   - Replace **<SID>** with your HANA SID.
   - Replace **<instance>** with your HANA instance number.
   - Replace **<address>**, **<netmask>** and **<device>** with the details of your primary virtual IP address.

2. Create a cluster constraint that places the VIP resource with the **SAPHanaController** resource on the HANA primary node:

```
[root]# pcs constraint colocation add rsc_vip_<SID>_HDB<instance>_primary \
with promoted cln_SAPHanaCon_<SID>_HDB<instance> 2000
```

   The constraint applies a score of **2000** instead of the default **INFINITY**. This softens the resource dependency and allows the virtual IP resource to stay active in the case when there is no promoted **SAPHanaController** resource. This way it is still possible to use tools like the SAP Management Console (MMC) or SAP Landscape Management (LaMa) that can reach this IP address to query the status information of the HANA instance.

Verification

1. Check the resource configuration of the virtual IP resource:

```
[root]# pcs resource config rsc_vip_RH1_HDB02_primary
Resource: rsc_vip_RH1_HDB02_primary (class=ocf provider=heartbeat type=IPaddr2)
  Attributes: rsc_vip_RH1_HDB02_primary-instance_attributes
    cidr_netmask=32
    ip=192.168.1.100
    nic=eth0
  Operations:
    monitor: rsc_vip_RH1_HDB02_primary-monitor-interval-10s
      interval=10s timeout=20s
    start: rsc_vip_RH1_HDB02_primary-start-interval-0s
      interval=0s timeout=20s
    stop: rsc_vip_RH1_HDB02_primary-stop-interval-0s
      interval=0s timeout=20s
```

2. Check that the constraint is defined correctly:

```
[root]# pcs constraint colocation
Colocation Constraints:
  Started resource 'rsc_vip_RH1_HDB02_primary' with Promoted resource
'cln_SAPHanaCon_RH1_HDB02'
    score=2000
```

5.9. ADDING A SECONDARY (READ-ENABLED) VIRTUAL IP ADDRESS

To support the Active/Active (read-enabled) secondary setup you must add a second virtual IP to provide client access to the secondary SAP HANA instance.

Configure additional rules to ensure that the second virtual IP is always associated with a healthy SAP HANA instance, maximizing client access and availability.

- Normal operation
  When both primary and secondary SAP HANA instances are active and replication is in sync, the second virtual IP is assigned to the secondary node.

- Secondary unavailable or out of sync
  If the secondary instance is down or replication is not in sync, the virtual IP moves to the primary node. It automatically returns to the secondary node as soon as system replication is back in sync.

- Failover scenario
  In case the cluster triggers a takeover, the virtual IP remains on the same node. After the former primary node takes over the secondary role and replication is in sync again, this VIP shifts there accordingly.

Prerequisites

- You have set **operationMode=logreplay_readaccess** when registering the secondary SAP HANA instance for system replication with the primary.

Procedure

1. Use the appropriate resource agent for managing the virtual IP address based on the platform on which the HA cluster is running. Adjust the parameters according to the resource agent you are using. Create the cluster resource for the secondary virtual IP, for example using the **IPaddr2** agent:

   ```
   [root]# pcs resource create rsc_vip_<SID>_HDB<instance>_readonly \
   ocf:heartbeat:IPaddr2 ip=<address> cidr_netmask=<netmask> nic=<device>
   ```

   - Replace **<SID>** with your HANA SID.
   - Replace **<instance>** with your HANA instance number.
   - Replace **<address>**, **<netmask>** and **<device>** with the details of your read-only secondary virtual IP address.

2. Create a location constraint rule to ensure that the secondary virtual IP is assigned to the secondary instance during normal operations:

   ```
   [root]# pcs constraint location rsc_vip_<SID>_HDB<instance>_readonly \
   rule score=INFINITY master-rsc_SAPHanaCon_<SID>_HDB<instance> eq 100 \
   and hana_<sid>_clone_state eq DEMOTED
   ```

   - Replace **<SID>** with your HANA SID.
   - Replace **<sid>** with the lower-case HANA SID.

- Replace **<instance>** with your HANA instance number.

3. **Create a location constraint rule to ensure that the secondary virtual IP runs on the primary instance as an alternative whenever necessary:**

```
[root]# pcs constraint location rsc_vip_<SID>_HDB<instance>_readonly \
rule score=2000 master-rsc_SAPHanaCon_<SID>_HDB<instance> eq 150 \
and hana_<sid>_clone_state eq PROMOTED
```

Verification

- **Check that the constraints are part of the cluster configuration:**

```
[root]#  pcs constraint location
Location Constraints:
  resource 'rsc_vip_RH1_HDB02_readonly'
    Rules:
      Rule: boolean-op=and score=INFINITY
        Expression: master-rsc_SAPHanaCon_RH1_HDB02 eq 100
        Expression: hana_rh1_clone_state eq DEMOTED
  resource 'rsc_vip_RH1_HDB02_readonly'
    Rules:
      Rule: boolean-op=and score=2000
        Expression: master-rsc_SAPHanaCon_RH1_HDB02 eq 150
        Expression: hana_rh1_clone_state eq PROMOTED
```

Additional resources

- SAP HANA System Replication - Active/Active (Read Enabled)
- **pcs(8)** man page on your system

CHAPTER 6. TESTING THE SETUP

Test your new HANA HA cluster thoroughly before you enable it for production workloads.

Enhance the basic example test cases with your specific requirements.

6.1. DETECTING THE SYSTEM REPLICATION STATE CHANGES

You must monitor the sync state information in logs and cluster attributes when disrupting the system replication, to test the correct functionality of the **HanaSR** HA/DR provider.

In this test, the primary instance is used for monitoring the system replication status and verifying the log messages. The secondary instance freezes the secondary instance indexserver process, to simulate a replication issue while the primary remains fully intact.

Prerequisites

- You have configured the mandatory **HanaSR** HA/DR provider.
- Your HANA instances are in a healthy state on all cluster nodes and the system replication is in sync.

Procedure

1. As user **<sid>adm** go to the HANA Python directory on the primary instance and check the current system replication state. Verify that it is **ACTIVE** and fully synced:

```
rh1adm $ cdpy; python systemReplicationStatus.py
```

2. Verify that the **srHook** and **srPoll** cluster attributes are both **SOK** in the attributes summary of the secondary node. Run this command as the **root** user on any node in a separate terminal to keep track of the attribute changes:

```
[root]# watch SAPHanaSR-showAttr
...
Site lpt       lss mns   opMode    srHook srMode srPoll srr
--------------------------------------------------------------
DC2  10        4   node2 logreplay SOK    sync   SOK    S
DC1  1745833771 4   node1 logreplay PRIM   sync   PRIM   P
...
```

   Use the **watch** command to run the following command in a loop at a default interval of 2 seconds.

3. On the secondary instance, get the process ID (**PID**) of the **hdbindexserver** process, for example from the **PID** column of the **HDB** info output, as user **<sid>adm**:

```
rh1adm $ HDB info
```

4. On the secondary instance, simulate a hanging **hdbindexserver** process by sending the **STOP** signal to the process. This freezes the process and blocks it from communicating and syncing the instance between the nodes. Ensure to execute this on the secondary instance, since the result is not the same when done on the primary:

```
rh1adm $ kill -STOP <PID>
```

Verification

1. **On the primary instance, watch the system replication status for the change:**

```
rh1adm $ cdpy; watch python systemReplicationStatus.py
...
|Database |Host  |Port  |Service Name |...|Secondary     |Replication |Replication |Replication
|Secondary    |
|        |      |      |             |...|Active Status |Mode        |Status      |Status Details              |Fully
Synced |
|-------- |----- |----- |------------ |...|------------- |----------- |----------- |---------------------------- |---
--------- |
|SYSTEMDB |node1 |30201 |nameserver   |...|YES           |SYNC        |ACTIVE      |
|       True |
|RH1      |node1 |30207 |xsengine     |...|YES           |SYNC        |ACTIVE      |
|       True |
|RH1      |node1 |30203 |indexserver  |...|YES           |SYNC        |ERROR       |Log shipping
timeout occurred |       False |

status system replication site "2": ERROR
overall system replication status: ERROR

...
```

The replication status changes to ERROR for the indexserver service after a bit. It can take a while to react on an idle instance, wait a minute or more.

2. **On the primary instance, as the <sid>adm user, check the HANA nameserver log for the related messages:**

```
rh1adm $ cdtrace; grep -he 'HanaSR.srConnectionChanged.*' nameserver_*
ha_dr_provider   PythonProxyImpl.cpp(01113) : calling HA/DR provider
HanaSR.hookDRConnectionChanged(hostname=node1, port=34203, volume=3,
service_name=indexserver, database=RH1, status=11, database_status=11,
system_status=11, timestamp=*****, is_in_sync=0, system_is_in_sync=0, reason=,
siteName=DC2)
ha_dr_HanaSR     HanaSR.py(00056) : HanaSR 1.001.1 HanaSR.srConnectionChanged
method called with Dict={'hostname': 'node1', 'port': '34203', 'volume': 3, 'service_name':
'indexserver', 'database': 'RH1', 'status': 11, 'database_status': 11, 'system_status': 11,
'timestamp': '*****', 'is_in_sync': False, 'system_is_in_sync': False, 'reason': '', 'siteName':
'DC2'}
ha_dr_HanaSR     HanaSR.py(00065) : HanaSR HanaSR.srConnectionChanged
system_status=11 SID=RH1 in_sync=False reason=
ha_dr_HanaSR     HanaSR.py(00091) : HanaSR.srConnectionChanged() CALLING CRM:
<sudo /usr/sbin/crm_attribute -n hana_rh1_site_srHook_DC2  -v SFAIL -t crm_config -s
SAPHanaSR> ret_code=0
```

The nameserver log contains the event that HANA triggers with details. It also includes the sudo command that the HanaSR hook script runs to update the srHook cluster attribute.

3. Verify that both cluster attributes for the system replication status, **srHook** and **srPoll**, show the **SFAIL** status of the secondary. Run as the **root** user on any node or use the open terminal from procedure step 3 to watch the changes:

```
[root]# SAPHanaSR-showAttr
...
Site lpt        lss mns   opMode    srHook srMode srPoll srr
-------------------------------------------------------------
DC2  10         4   node2 logreplay SFAIL  sync   SFAIL  S
DC1  1745833771 4   node1 logreplay PRIM   sync   PRIM   P
...
```

4. On the secondary instance, unblock the previously frozen **hdbindexserver PID** to enable it again:

```
rh1adm $ kill -CONT <PID>
```

5. Repeat steps 1-3 to verify that the system replication recovers fully after a bit. The cluster does not trigger any actions during this test since the resources remain running.

Additional resources

watch(1) and **kill(1)** man pages on your system

6.2. TRIGGERING THE INDEXSERVER CRASH RECOVERY

Test the functionality of the **ChkSrv** HA/DR provider by simulating the crash of an **hdbindexserver** process. You can run this on the primary or on the secondary instance. The exact recovery actions depend on the overall configuration. The following steps demonstrate the activity when using **action_on_lost = stop**.

Prerequisites

- You have configured the **ChkSrv** HA/DR provider. Skip this test if you have not configured this optional hook.
- Your HANA instances have a healthy HANA system replication.
- You have no failures in the cluster status.

Procedure

1. Use a separate terminal to monitor the HANA instance processes as user **<sid>adm**:

```
rh1adm $ watch "sapcontrol -nr ${TINSTANCE} -function GetProcessList | column -s ',' -t"
```

2. In another terminal, kill the **hdbindexserver** process:

```
rh1adm $ kill <PID>
```

Verification

1. **Check the dedicated HANA trace log on the same instance and identify the event and related action, as user <sid>adm:**

```
rh1adm $ cdtrace; less nameserver_chksrv.trc
...
ChkSrv srServiceStateChanged method called with SAPSYSTEMNAME=RH1
srv:indexserver-30203-stopping-yes db:RH1-3-yes daem:yes
LOST: indexserver event looks like a lost indexserver (status=stopping)
LOST: stop instance. action_on_lost=stop
ChkSrv version 1.001.1. Method srServiceStateChanged method called.
...
```

2. **Check the cluster status for resource failure information as user root:**

```
[root]# pcs status --full
...

Failed Resource Actions:
  * rsc_SAPHanaCon_RH1_HDB02_start_0 on node1 'not running' (7): call=61,
status='complete', ...

...
```

3. **Check the system log for the related action on the cluster side as user root:**

```
[root]# grep rsc_SAPHanaCon_RH1_HDB02 /var/log/messages
...
SAPHanaController(rsc_SAPHanaCon_RH1_HDB02)[149199]: INFO: ##-2-## DEC:
PRIMDEFECT (in DEMOTED status)
SAPHanaController(rsc_SAPHanaCon_RH1_HDB02)[149206]: INFO: ##-2-## RA ==== end
action monitor_clone with rc=7 (1.2.7) (3s; times=0m0.051s 0m0.079s 0m1.325s
0m1.137s)====
pacemaker-controld[1694]: notice: Result of monitor operation for
rsc_SAPHanaCon_RH1_HDB02 on node1: not running
pacemaker-controld[1694]: notice: Transition 142 action 18
(rsc_SAPHanaCon_RH1_HDB02_monitor_61000 on node1): expected 'ok' but got 'not
running'
pacemaker-attrd[1692]: notice: Setting last-failure-
rsc_SAPHanaCon_RH1_HDB02#monitor_61000[node1] in instance_attributes: (unset) ->
1746703980
pacemaker-attrd[1692]: notice: Setting fail-count-
rsc_SAPHanaCon_RH1_HDB02#monitor_61000[node1] in instance_attributes: (unset) -> 1
pacemaker-attrd[1692]: notice: Setting master-rsc_SAPHanaCon_RH1_HDB02[node2] in
instance_attributes: 100 -> 145
pacemaker-schedulerd[1693]: warning: Unexpected result (not running) was recorded for
monitor of rsc_SAPHanaCon_RH1_HDB02:0 on node1 at ...
pacemaker-schedulerd[1693]: notice: Actions: Recover    rsc_SAPHanaCon_RH1_HDB02:0
(            Unpromoted node1 )
pacemaker-controld[1694]: notice: Initiating monitor operation
rsc_SAPHanaCon_RH1_HDB02_monitor_59000 on node2
pacemaker-controld[1694]: notice: Initiating stop operation
rsc_SAPHanaCon_RH1_HDB02_stop_0 locally on node1
pacemaker-controld[1694]: notice: Requesting local execution of stop operation for
rsc_SAPHanaCon_RH1_HDB02 on node1
...
```

The next **SAPHanaController** resource monitor reports the unexpectedly stopped HANA instance as a failure and initiates the recovery steps according to the configuration. If **PREFER_SITE_TAKEOVER** is enabled and the test was run on the primary instance, it triggers a HANA takeover to the secondary instance.

Next steps

- Clear any failure notifications from the cluster that might be there from previous testing. For more information see Cleaning up the failure history.
- When necessary, depending on the configuration, manually reregister the stopped former primary HANA instance and start it using HANA tools. For more information, refer to Registering the former primary after a takeover.

Additional resources

watch(1) and **kill(1)** man pages on your system

6.3. TRIGGERING A HANA TAKEOVER USING CLUSTER COMMANDS

Use the cluster command to move the promoted resource to the other node and manually test the planned takeover of the primary instance to the secondary.

Prerequisites

- Your HANA instances have a healthy HANA system replication.
- You have no failures in the cluster status.

Procedure

- Switch the primary instance to the secondary node. Run the cluster command as user **root** on any node:

```
[root]# pcs resource move cln_SAPHanaCon_<SID>_HDB<instance>
Location constraint to move resource 'cln_SAPHanaCon_RH1_HDB02' has been created
Waiting for the cluster to apply configuration changes...
Location constraint created to move resource 'cln_SAPHanaCon_RH1_HDB02' has been
removed
Waiting for the cluster to apply configuration changes...
resource 'cln_SAPHanaCon_RH1_HDB02' is promoted on node 'node2'
```

Verification

- Verify that the **SAPHanaController** resource is now promoted on the other node:

```
[root]# pcs status --full
...
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02]
(promotable):
    * rsc_SAPHanaCon_RH1_HDB02       (ocf:heartbeat:SAPHanaController):       Promoted
node2
    * rsc_SAPHanaCon_RH1_HDB02       (ocf:heartbeat:SAPHanaController):       Stopped
...
```

-

The status of the previous primary instance depends on the **AUTOMATED_REGISTER** parameter of the **SAPHanaController** resource. The instance stops until manual intervention when **AUTOMATED_REGISTER** is **false**, otherwise the instance restarts automatically and reregisters as the new secondary instance.

Next steps

- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.
- When necessary, depending on the configuration, manually reregister the stopped former primary HANA instance and start it using HANA tools. For more information, refer to Registering the former primary after a takeover.

Additional resources

- Is it possible to use SAP HANA "Takeover with Handshake" option with the HA Solutions for managing HANA System Replication?
- **pcs(8)** man page on your system

6.4. TRIGGERING THE SAPHANAFILESYSTEM FAILURE ACTION

Block the write access to the monitored directory to test the correct behavior of the **SAPHanaFilesystem** resource. You can test this on both instances. Only the primary instance triggers a failure and recovery action. On the secondary node the resource does not trigger an action.

Prerequisites

- You have configured the **SAPHanaFilesystem** resource. Skip this test if you have not configured this optional resource.

Procedure

1. Go to the hidden directory which the **SAPHanaFilesystem** resource uses to test read and write filesystem access:

```
[root]# cd /hana/shared/<SID>/.heartbeat_SAPHanaFilesystem/<node>
```

2. Set the existing **test** file to become immutable, which prevents write access by the resource monitors:

```
[root]# chattr +i test
```

3. Verify the behavior during the simulated failure.

   a. You can check the **/var/log/messages** file for the related log message if the resource action is set to **ignore**:

```
[root]# grep ON_FAIL_ACTION /var/log/messages
... node1 SAPHanaFilesystem(rsc_SAPHanaFil_RH1_HDB02)[715184]: INFO: -2- RA
monitor() ON_FAIL_ACTION=ignore => ignore FS error, do not create poison pill file
```

b. You can verify the fence behavior if the resource action is set to **fence**:

```
[root]# pcs status --full
...

Failed Resource Actions:
  * rsc_SAPHanaFil_RH1_HDB02_stop_0 on node1 'error' (1): ...

Pending Fencing Actions:
  * reboot of node1 pending: client=pacemaker-controld.1694, origin=node2
```

4. You must remove this blocker again after the test. If the node is fenced, do this after the node is running again:

```
[root]# chattr -i test
```

Next steps

- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.
- When necessary, depending on the configuration, manually reregister the stopped former primary HANA instance and start it using HANA tools. For more information, refer to Registering the former primary after a takeover.

Additional resources

chattr(1) and **pcs(8)** man pages on your system

6.5. CRASHING THE NODE WITH THE PRIMARY INSTANCE

Simulate the crash of the cluster node on which the primary instance is running to test the behavior of your HANA cluster resources.

Prerequisites

- Your HANA instances have a healthy HANA system replication.
- You have no failures in the cluster status.

Procedure

- Trigger a crash on the primary node. This command immediately causes a crash of the node with no further warning:

```
[root]# echo c > /proc/sysrq-trigger
```

Verification

- The cluster on the secondary node fences the primary node:

```
[root]# pcs status --full
```

```
...
Pending Fencing Actions:
  * reboot of node1 pending: client=pacemaker-controld.1685, origin=node2
...
```

- The secondary takes over and becomes promoted as the new primary.
- The fenced former primary node recovers according to your fencing and **SAPHanaController** resource configuration.

Next steps

- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.
- When necessary, depending on the configuration, manually reregister the stopped former primary HANA instance and start it using HANA tools. For more information, refer to Registering the former primary after a takeover.

Additional resources

How to use the SysRq Facility to collect information from a RHEL system

6.6. CRASHING THE NODE WITH THE SECONDARY INSTANCE

Simulate the crash of the cluster node on which the secondary instance is running to test the behavior of your HANA cluster resources.

Procedure

- Trigger a crash on the secondary node. This command immediately causes a crash of the node with no further warning:

```
[root]# echo c > /proc/sysrq-trigger
```

Verification

- The cluster on the primary node fences the secondary node:

```
[root]# pcs status --full
...
Pending Fencing Actions:
  * reboot of node2 pending: client=pacemaker-controld.1694, origin=node1
...
```

- The primary instance remains running while the secondary node restarts and recovers. The fenced node recovery depends on your fencing configuration.

Next steps

- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.

Additional resources

How to use the SysRq Facility to collect information from a RHEL system

6.7. STOPPING THE PRIMARY INSTANCE USING SAP COMMANDS

Test the behavior of the cluster when you manage the primary HANA instance outside of the cluster using HANA commands.

Since the cluster is not aware of the execution of HANA commands, it detects the change as a failure and triggers the configured recovery actions.

Prerequisites

- Your HANA instances have a healthy HANA system replication.
- You have no failures in the cluster status.

Procedure

- Stop the primary instance as the **<sid>adm** user outside of the cluster:

```
rh1adm $ HDB stop
```

Verification

- The cluster notices the stopped instance as a failure and initiates the recovery of the primary instance:

```
[root]# pcs status --full
...
Migration Summary:
  * Node: node1 (1):
    * rsc_SAPHanaCon_RH1_HDB02: migration-threshold=5000 fail-count=1 last-failure=...

Failed Resource Actions:
  * rsc_SAPHanaCon_RH1_HDB02_monitor_61000 on node1 'not running' ...
```

  If you configured and enabled both the **PREFER_SITE_TAKEOVER** and **AUTOMATED_REGISTER** parameters in the**SAPHanaController** resource, the cluster triggers a HANA takeover to the secondary instance and automatically registers the failed primary as the new secondary. Otherwise it recovers the failed primary according to your configuration.

Next steps

- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.
- When necessary, depending on the configuration, manually reregister the stopped former primary HANA instance and start it using HANA tools. For more information, refer to Registering the former primary after a takeover.

6.8. STOPPING THE SECONDARY INSTANCE USING SAP COMMANDS

Test the behavior of the cluster when you manage the secondary HANA instance outside of the cluster using HANA commands.

Since the cluster is not aware of the execution of HANA commands, it detects the change as a failure and triggers the configured recovery actions.

Prerequisites

- You have no failures in the cluster status.

Procedure

- Stop the secondary instance as the **<sid>adm** user outside of the cluster:

```
rh1adm $ HDB stop
```

Verification

- The cluster notices the stopped instance as a failure and recovers the secondary:

```
[root]# pcs status --full
...
Migration Summary:
  * Node: node2 (2):
    * rsc_SAPHanaCon_RH1_HDB02: migration-threshold=5000 fail-count=1 last-failure=...

Failed Resource Actions:
  * rsc_SAPHanaCon_RH1_HDB02_monitor_61000 on node2 'not running' ...
```

Next steps

- Clear any failure notifications from the cluster that may be there from previous testing. For more information see Cleaning up the failure history.

CHAPTER 7. FINISHING THE SETUP

Ensure that the final setup is complete and the systems and resources are healthy, then you can enable the environment for production workloads.

7.1. ENABLING THE AUTOMATIC REGISTRATION OF HANA AFTER A TAKEOVER

If you want a previously failed primary instance to automatically recover as a fully functional secondary instance without manual verification of the data consistency, you can enable the **SAPHanaController** resource to re-register the instance right after a takeover.

This enables the previously failed primary instance to continue the HANA system replication and automatically take over again in the event of a new failure of the new primary instance.

Your HANA operator must decide if they first require to manually check the health of the previously failed instance and re-register the instance afterwards, or if the priority is on a faster automatic recovery of the full high availability.

Procedure

- Update the **SAPHanaController** resource and override the default **AUTOMATED_REGISTER**:

```
[root]# pcs resource update rsc_SAPHanaCon_<SID>_HDB<instance>
AUTOMATED_REGISTER=true
```

Verification

- Check that **AUTOMATED_REGISTER** is set to**true**:

```
[root]# pcs resource config rsc_SAPHanaCon_RH1_HDB02
Resource: rsc_SAPHanaCon_RH1_HDB02 (class=ocf provider=heartbeat
type=SAPHanaController)
  Attributes: rsc_SAPHanaCon_RH1_HDB02-instance_attributes
    AUTOMATED_REGISTER=true
    DUPLICATE_PRIMARY_TIMEOUT=7200
    InstanceNumber=02
    PREFER_SITE_TAKEOVER=true
    SID=RH1
...
```

> **WARNING**
>
> Setting **AUTOMATED_REGISTER** to **true** can potentially increase the risk of data loss or corruption. If the HA cluster triggers a takeover when the data on the secondary HANA instance is not fully in sync, the automatic registration of the old primary HANA instance as the new secondary HANA instance results in data loss on this instance and any data that was not synced before the takeover occurred is lost as well.
>
> For more information, see the article on the SAP Technology Blog for Members: Be Prepared for Using Pacemaker Cluster for SAP HANA – Part 2: Failure of Both Nodes.

Additional resources

pcs(8) man page on your system

7.2. REVIEWING THE FINAL CLUSTER STATE

After the configuration of the 2-node cluster for a scale-up HANA system replication setup, the status looks like in the following example.

Your setup may slightly deviate from the example, if you did not configure the optional **SAPHanaFilesystem** resources and the optional second virtual IP for the read-only access on the secondary instance.

Also, you can decide if you want to disable the cluster service, so that it does not start automatically on system boot. This requires manual intervention on every system boot, but allows you more control and supervision for the startup.

```
[root]# pcs status --full
...

Cluster name: node1-node2-cluster
Cluster Summary:
  * Stack: corosync (Pacemaker is running)
  * Current DC: node2 (2) (version 2.1.7-5.2.el9_4-0f7f88312) - partition with quorum
  * Last updated: Thu May  8 13:49:46 2025 on node1
  * Last change:  Thu May  8 13:49:14 2025 by root via root on node1
  * 2 nodes configured
  * 9 resource instances configured

Node List:
  * Node node1 (1): online, feature set 3.19.0
  * Node node2 (2): online, feature set 3.19.0

Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]:
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node1
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node2
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02] (promotable):
```

```
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Promoted node1
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Unpromoted node2
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02]:
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started node1
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started node2
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):         Started node1
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):         Started node2

Node Attributes:
  * Node: node1 (1):
    * hana_rh1_clone_state              : PROMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC1
    * hana_rh1_sra                      : -
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.078.00
    * hana_rh1_vhost                    : node1
    * master-rsc_SAPHanaCon_RH1_HDB02   : 150
  * Node: node2 (2):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC2
    * hana_rh1_sra                      : -
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.078.00
    * hana_rh1_vhost                    : node2
    * master-rsc_SAPHanaCon_RH1_HDB02   : 100

Migration Summary:

Tickets:

PCSD Status:
  node1: Online
  node2: Online

Daemon Status:
  corosync: active/enabled
  pacemaker: active/enabled
  pcsd: active/enabled
```

In a complete healthy setup the additional cluster attributes appear like in this example:

```
[root]# SAPHanaSR-showAttr
Global cib-update dcid prim sec sid topology
---------------------------------------------
global 0.550.2    2    DC1  DC2 RH1 ScaleUp

Resource                promotable
------------------------------------
cln_SAPHanaCon_RH1_HDB02 true
cln_SAPHanaTop_RH1_HDB02

Site lpt       lss mns   opMode    srHook srMode srPoll srr
```

```
-----------------------------------------------------------------
DC2  30         4   node2 logreplay SOK    sync   SOK    S
DC1  1746780164 4   node1 logreplay PRIM   sync   PRIM   P

Host  clone_state roles                        score site srah version     vhost
-------------------------------------------------------------------------------
node1 PROMOTED    master1:master:worker:master 150   DC1  -    2.00.078.00 node1
node2 DEMOTED     master1:master:worker:master 100   DC2  -    2.00.078.00 node2
```

CHAPTER 8. MAINTENANCE PROCEDURES

You must apply specific steps to ensure that the cluster does not cause unplanned impact so that you can perform maintenance of different components ofSAP HANA system replication HA environments.

Use maintenance procedures to keep your cluster in a healthy state during planned change activity or to restore the health after unplanned incidents.

8.1. CLEANING UP THE FAILURE HISTORY

Clear any failure notifications from the cluster that may be there from previous testing. This resets the failure counters and the migration thresholds.

Procedure

1. Clean up resource failures:

```
[root]# pcs resource cleanup
```

2. Clean up the STONITH failure history:

```
[root]# pcs stonith history cleanup
```

Verification

- Check the overall cluster status and confirm that no failures are displayed anymore:

```
[root]# pcs status --full
```

- Check that the stonith history for fencing actions has 0 events:

```
[root]# pcs stonith history
```

Additional resources

pcs(8) man page on your system

8.2. TRIGGERING A HANA TAKEOVER USING CLUSTER COMMANDS

Use the cluster control to execute a simple takeover of the primary instance to the other node.

For detailed steps, refer to the section Testing the setup - Triggering a HANA takeover using cluster commands.

8.3. UPDATING THE OPERATING SYSTEM AND HA CLUSTER COMPONENTS

For updates or offline changes on the HA cluster, the operating system or even the system hardware, you must follow the Recommended Practices for Applying Software Updates to a RHEL High Availability or Resilient Storage Cluster.

8.4. PERFORMING MAINTENANCE ON THE SAP HANA INSTANCES

For any kind of maintenance of applications or other components that the HA cluster manages, you must enable the cluster **maintenance** mode to prevent the cluster from any interference during the maintenance.

During the update of your HANA instances, the cluster remains running, but is not actively monitoring resources or taking any actions. After the change on the HANA instance is done, it is vital to refresh the cluster resource status and verify that the detected resource states are all correct. Only then can the maintenance mode be disabled again without unexpected cluster actions.

If you need to stop the cluster for the maintenance activity, ensure that you set **maintenance** mode first, then stop and start the cluster on the node as required for the HANA maintenance.

Prerequisites

- You have configured the Pacemaker cluster to manage the HANA system replication.

Procedure

1. Set **maintenance** mode for the entire cluster:

```
[root]# pcs property set maintenance-mode=true
```

   Setting maintenance for the whole cluster ensures that no activity during the maintenance phase can trigger cluster actions and impact the HANA update process.

2. Verify that the cluster resource management is fully disabled:

```
[root]# pcs status
...

              *** Resource management is DISABLED ***
  The cluster will not attempt to start, stop or recover services

Node List:
  * Online: [ node1 node2 ]

Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]
(maintenance):
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node2
(maintenance)
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node1
(maintenance)
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02]
(promotable, maintenance):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Unpromoted
node2 (maintenance)
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Promoted
node1 (maintenance)
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02] (maintenance):
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started node2
(maintenance)
```

```
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started node1
(maintenance)
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):        Started node1
(maintenance)
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):        Started node2
(maintenance)

...
```

3. Update the HANA instances using the SAP procedure. If you have to perform a takeover during the HANA update, you can use the SAP HANA Takeover with Handshake option. For more information see also Is it possible to use SAP HANA "Takeover with Handshake" option with the HA Solutions for managing HANA System Replication?.
   If you stop the cluster in this step, ensure that you start it again before you proceed with the next steps. Keep the maintenance mode enabled.

4. After the HANA update, verify that the HANA system replication is working correctly. Use the **systemReplicationStatus.py** script to show the status of the HANA system replication on the primary instance. Below is the example of post manual takeover to node2, during the maintenance:

```
[root]# su - <sid>adm

rh1adm $ cdpy; python systemReplicationStatus.py --sapcontrol=1 | grep -i
replication_status=
service/node2/30201/REPLICATION_STATUS=ACTIVE
service/node2/30207/REPLICATION_STATUS=ACTIVE
service/node2/30203/REPLICATION_STATUS=ACTIVE
site/1/REPLICATION_STATUS=ACTIVE
overall_replication_status=ACTIVE
```

   Before you proceed, ensure that the system replication is healthy and reported as **ACTIVE**.

5. Refresh all cluster resources to execute one monitor operation and update their status:

```
[root]# pcs resource refresh
Waiting for 1 reply from the controller
... got reply (done)
```

   It is crucial that the HANA resources update cluster and node attributes, to reflect the new HANA system replication status. It ensures that the cluster has the correct information and does not trigger recovery actions due to incorrect status information, after the maintenance stops.

6. Check the cluster status and verify the resource status and main HANA resource score attribute. All resources must show as **Started** and the promotable resources must show as **Unpromoted** on all nodes:

```
[root]# pcs status
...
Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]
(maintenance):
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):      Started node2
(maintenance)
```

```
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node1
(maintenance)
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02]
(promotable, maintenance):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Unpromoted
node2 (maintenance)
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Unpromoted
node1 (maintenance)
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02] (maintenance):
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started node2
(maintenance)
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started node1
(maintenance)
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):        Started node1
(maintenance)
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):        Started node2
(maintenance)

...
```

7. **Check the cluster attributes and verify that srHook, roles and score attributes are in the correct new state:**

```
[root]# SAPHanaSR-showAttr
...

Site lpt       lss mns   opMode    srHook srMode srPoll srr
-------------------------------------------------------------
DC2  1746793335 4   node2 logreplay PRIM   sync   SOK    S
DC1  30         4   node1 logreplay SOK    sync   PRIM   P

Host  clone_state roles                    score site sra srah version     vhost
-----------------------------------------------------------------------------------
node1 DEMOTED     master1:master:worker:master 100   DC1  -   -    2.00.078.00 node1
node2 DEMOTED     master1:master:worker:master 150   DC2  -   -    2.00.078.00 node2
```

   - **The srHook is PRIM on the node that is running the primary instance and it shows SOK on the correct secondary.**
   - **The score is 150 for the node where the primary is running and is 100 on the other.**

8. **When the checks of steps 6 and 7 show the landscape in the expected healthy state, you can remove the maintenance mode of the cluster again:**

```
[root]# pcs property set maintenance-mode=
```

When you lift the maintenance it triggers a monitor run of all resources again. The cluster updates the status of the promotable resources to Promoted and Unpromoted in correspondence to the location of the primary and secondary instances. The resources now also update the srPoll attribute again to match the srHook attribute value.

Additional resources

- Update SAP HANA Systems Running in a System Replication Setup

- **pcs(8)** man page on your system

8.5. REGISTERING THE FORMER PRIMARY AFTER A TAKEOVER

When you configure **AUTOMATED_REGISTER=false** in the **SAPHanaController** resource, which is the default, you must manually register the former primary instance as the new secondary after takeover and start it. Otherwise, the unregistered instance remains stopped.

Procedure

1. Register the former primary as the new secondary. Run as user **<sid>adm** on the stopped former primary instance:

```
rh1adm $ hdbnsutil -sr_register --remoteHost=<node> \
--remoteInstance=${TINSTANCE} --replicationMode=sync \
--operationMode=logreplay --name=<DC>
```

   - Replace **<node>** with the new primary instance host, for example node2 if there was a takeover from node1 to node2.
   - Replace **<DC>** with your new secondary HANA site name, for example DC1 if node1 is to be registered as a secondary.
   - Choose the values for **replicationMode** and **operationMode** according to your requirements for the system replication.
   - **TINSTANCE** is an environment variable that is set automatically for user **<sid>adm** by reading the HANA instance profile. The variable value is the HANA instance number.

2. Start the secondary HANA instance. Run as **<sid>adm** on the new secondary instance node:

```
rh1adm $ HDB start
```

Verification

- On the current primary instance, show the current status of the re-established HANA system replication. Below is the example after a takeover and the secondary instance is re-registered on node1:

```
rh1adm $ cdpy; python systemReplicationStatus.py
|Database |Host     |Port  |Service Name |Volume ID |Site ID |Site Name |Secondary
|Secondary |Secondary |Secondary |Secondary     |Replication |Replication |Replication
|Secondary    |
|         |         |      |             |          |        |          |Host      |Port      |Site ID   |Site Name |Active
Status |Mode        |Status      |Status Details |Fully Synced |
|-------- |-------- |----- |------------ |--------- |------- |--------- |----------|--------- |--------- |-
------------ |----------- |----------- |-------------- |------------ |
|SYSTEMDB |node2    |30001 |nameserver   |        1 |      2 |DC2       |node1     |    30001 |
1 |DC1       |YES           |SYNC        |ACTIVE      |              |        True |
|RH1      |node2    |30007 |xsengine     |        2 |      2 |DC2       |node1     |    30007 |        1
|DC1       |YES           |SYNC        |ACTIVE      |              |        True |
|RH1      |node2    |30003 |indexserver  |        3 |      2 |DC2       |node1     |    30003 |        1
|DC1       |YES           |SYNC        |ACTIVE      |              |        True |
```

```
status system replication site "1": ACTIVE
overall system replication status: ACTIVE

Local System Replication State
~~~~~~~~~~

mode: PRIMARY
site id: 2
site name: DC2
```

CHAPTER 9. TROUBLESHOOTING

9.1. THE SRHOOK CLUSTER ATTRIBUTE VALUE IS INCORRECT

When the **srHook** attribute value does not match the actual HANA system replication status, it can lead to unexpected behavior in the cluster when a failure of the primary instance occurs.

Check and correct your sudo configuration when the **srHook** attribute of the secondary instance and the HANA system replication status do not match:

- The **srHook** cluster attribute of the secondary is empty.
- The **srHook** cluster attribute of the secondary is set to **SOK** while the HANA system replication is not healthy.
- The **srHook** cluster attribute of the secondary is set to **SFAIL** while the system replication is in **ACTIVE** state.

The primary instance receives the events of HANA system replication changes and stores the result as a cluster attribute for the secondary instance.

Procedure

1. Check for **crm_attribute** update errors in the **secure** log, since the command is executed using **sudo**. The log shows the command that the hook script tries to execute, but potentially fails. Check on the primary instance node for an error like **command not allowed**, like in this example:

```
[root]# grep crm_attribute /var/log/secure
... rh1adm : command not allowed ; PWD=/hana/shared/RH1/HDB02/node1 ; USER=root ;
COMMAND=/usr/sbin/crm_attribute -n hana_rh1_site_srHook_DC2 -v SFAIL -t crm_config -s
SAPHanaSR
```

2. Compare the logged **COMMAND** to your **sudoers** configuration. Check thoroughly and fix the **sudoers** file, so that you have a sudo entry that matches the command. As a temporary measure you can ensure that the sudo entry as such works by simplifying it with a wildcard to exclude typos in the command parameters as the cause:

```
[root]# cat /etc/sudoers.d/20-saphana
<sid>adm ALL=(ALL) NOPASSWD: /usr/sbin/crm_attribute *
Defaults:<sid>adm !requiretty
```

   Replace **<sid>** with your lower-case HANA SID.

3. Verify that the command path is correct:

```
[root]# ls /usr/sbin/crm_attribute
/usr/sbin/crm_attribute
```

4. Fix the sudo configuration. For more information, see Configuring the HanaSR HA/DR provider for the srConnectionChanged() hook method.

5. Repeat any fixing steps on the other node. The sudo configuration must be identical on all instances.

Additional resources

sudo(8) and **visudo(8)** man pages on your system

9.2. THE HANA INSTANCE DOES NOT START AFTER HOOK CHANGES

You recently made changes in the **global.ini** in a HA/DR provider section and the HANA instance does not start anymore.

Procedure

1. Go to the HANA trace logs directory, as the **<sid>adm** user:

```
rh1adm $ cdtrace
```

2. Check for errors related to the HA/DR providers in the HANA nameserver alert log:

```
rh1adm $ grep ha_dr_provider nameserver_alert_*.trc
... ha_dr_provider   PythonProxyImpl.cpp(00145) : import of hanasr failed: No module named
'hanasr'
... ha_dr_provider   HADRProviderManager.cpp(00100) : could not load HA/DR Provider
'hanasr' from /usr/share/sap-hana-ha/
```

3. Identify the root cause, for example a misspelled HA/DR **provider** name or a wrong**path**. Check the path and the hook script name. In this example the HA/DR provider name **hanasr** is not matching the hook script name **HanaSR**:

```
rh1adm $ ls /usr/share/sap-hana-ha/
ChkSrv.py  HanaSR.py  samples
```

4. Correct the **HanaSR** HA/DR provider configuration:

```
[ha_dr_provider_hanasr]
provider = HanaSR
path = /usr/share/sap-hana-ha/
execution_order = 1

[trace]
ha_dr_hanasr = info
```

- **provider** must match the name of the Python hook script. It is case-sensitive without the **.py** file suffix.
- **path** must be the path in which the hook script is stored.

9.3. A CLUSTER NODE IS REPORTED AS OFFLINE DURING MAINTENANCE

When **maintenance-mode** is set for the cluster, for example, for a HANA update, it can still notice issues between the nodes, but does not trigger recovery actions yet.

If you encounter such a situation, you must first fix the cause of the issue before you lift the maintenance mode.

Example: the corosync communication between the nodes is blocked

Both nodes report the other node as **offline**. If the **maintenance** mode is removed in this situation, the cluster tries to recover by fencing one node. This can have a severe impact on your ongoing HANA maintenance activity.

```
...
              *** Resource management is DISABLED ***
  The cluster will not attempt to start, stop or recover services

Node List:
  * Node hana1 (1): online, feature set 3.19.0
  * Node hana2 (2): UNCLEAN (offline)

Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02] (maintenance):
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started hana2
(UNCLEAN, maintenance)
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started hana1
(maintenance)
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02] (promotable,
maintenance):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Unpromoted hana2
(UNCLEAN, maintenance)
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):       Promoted hana1
(maintenance)
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02] (maintenance):
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started hana2
(UNCLEAN, maintenance)
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):       Started hana1
(maintenance)
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):        Started hana1 (maintenance)
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):        Started hana2 (UNCLEAN,
maintenance)

...
```

Identify the root cause of the issue, for example:

- Planned network maintenance on the cluster communication connection in parallel to your HANA maintenance.
- Unplanned outage of network connections due to network device failures or misconfiguration on operating system or network level.
- Firewall configuration blocking cluster communication ports.

Fix any issue to prevent the cluster from taking recovery measures when the cluster maintenance is removed.

APPENDIX A. COMPONENT OPTIONS

A.1. HA/DR PROVIDER OPTIONS FOR HANASR

Parameters that are available for the configuration of the **HanaSR** HA/DR provider are shown below:

Provider options	Required	Default	Description
provider = HanaSR	yes		The provider parameter must be set to the hook script name without the **.py** ending.
path = /usr/share/sap-hana-ha/	yes		The full path to the location of the hook script.
execution_order = 1	yes		Set to **1** to ensure that the **HanaSR** HA/DR provider is always executed before other optional hook scripts when an event occurs.

A.2. HA/DR PROVIDER OPTIONS FOR CHKSRV

Parameters that are available for the configuration of the **ChkSrv** HA/DR provider are shown below:

Provider options	Required	Default	Description
provider = ChkSrv	yes		The provider parameter must be set to the hook script name without the **.py** ending.
path = /usr/share/sap-hana-ha/	yes		The full path to the location of the hook script.
execution_order = 2	yes		Set to **2** or higher. The mandatory HanaSR provider must always be run first.

Provider options	Required	Default	Description
action_on_lost	no	ignore	Action to be triggered when a lost indexserver is identified. • **ignore**: do nothing, just write to tracefiles. • **stop**: execute **sapcontrol … StopSystem**. • **kill**: execute **HDB kill-<signal>**. The signal can be defined by the separate parameter **kill_signal**. • **fence**: trigger the cluster to fence the node on which the indexserver failed. This option uses the **SAPHanaSR-hookHelper** tool and requires a sudo entry for the tool for the **<sid>adm** user.
kill_signal	no	9	The signal that is used with the **kill** action.
stop_timeout	no	20s	How many seconds to wait for the **stop** action to return. This value must be greater than the value of the HANA parameter **forcedterminationtimeout**.

A.3. SAPHANATOPOLOGY RESOURCE PARAMETERS

Parameters that are available for the configuration of **SAPHanaTopology** resources are shown below:

Resource options	Required	Default	Description
SID	yes		SAP system identifier.
InstanceNumber	yes		Number of the SAP HANA instance.
HANA_CALL_TIMEOUT	no	120	Defines the timeout - how long a call to HANA to receive information can take, for example, when the resource agent executes **landscapeHostConfiguration.py**. There are some specific calls to HANA which have their own timeout values. If you increase the timeout for HANA calls of this resource you must also consider increasing the operation timeout values of the same resource.

A.4. SAPHANACONTROLLER RESOURCE PARAMETERS

Parameters that are available for the configuration of **SAPHanaController** resources are shown below:

Resource options	Required	Default	Description
SID	yes		SAP system identifier.
InstanceNumber	yes		Number of the SAP HANA instance.
DIR_EXECUTABLE	no		The fully qualified path to binaries such as **sapstartsrv** and **sapcontrol**. Specify this parameter if you have changed the SAP kernel directory location after the default SAP installation. SAP standard paths are searched by default.

Resource options	Required	Default	Description
DIR_PROFILE	no		The fully qualified path to the SAP START profile. Specify this parameter if you have changed the SAP profile directory location after the default SAP installation. SAP standard paths are searched by default.
HANA_CALL_TIMEOUT	no	120	Defines the timeout - how long a call to HANA to receive information can take, for example, when the resource agent runs **landscapeHostConfiguration.py**. There are some specific calls to HANA which have their own timeout values. If you increase the timeout for HANA calls of this resource you must also consider increasing the operation timeout values of the same resource.
INSTANCE_PROFILE	no		The name of the SAP HANA instance profile. Specify this parameter if you have changed the name of the SAP HANA instance profile after the default SAP installation. SAP standard paths are searched by default.

Resource options	Required	Default	Description
PREFER_SITE_TAKEOVER	no	false	Defines whether the resource agent should prefer to trigger takeover to the secondary site instead of restarting the primary site locally. However, a takeover is only triggered if the SAP HANA landscape status is on **ERROR**. For **FATAL** a local restart is initiated. ● **true**: Prefer to trigger the takeover to the secondary site. ● **false**: Prefer to restart locally.
AUTOMATED_REGISTER	no	false	Defines whether the resource agent automatically registers a former primary instance as a secondary during cluster resource start and if the **DUPLICATE_PRIMARY_TIMEOUT** condition is met. Registering an instance as a secondary initiates a data synchronisation from the primary and might overwrite local data. ● **false**: Manual intervention is required. ● **true**: The former primary is automatically registered as a secondary.

Resource options	Required	Default	Description
DUPLICATE_PRIMARY_TIMEOUT	no	7200	The time difference required between two primary time stamps (LPTs), in case a dual-primary situation occurs. If the difference between both nodes' last primary time stamps is less than **DUPLICATE_PRIMARY_TIMEOUT**, then the cluster holds one or both instances in a **WAITING** status. This gives an admin the chance to react to a failover. How the recovery proceeds after the **DUPLICATE_PRIMARY_TIMEOUT** has passed depends on the parameter **AUTOMATED_REGISTER**.

Resource options	Required	Default	Description
ON_FAIL_ACTION	no	proceed	Set this parameter to **fence** to let the resource trigger the node to be fenced. This functionality uses the **SAPHanaSR-hookHelper** tool to run the fence action. The **fence** option can improve the takeover and recovery time of a HANA instance if the instance stop usually takes a longer time. • **proceed**: continue with the usual action to a failed resource. • **fence**: react with a resource stop failure to trigger the cluster fencing action, but only if the **srHook** status is **SOK**. If the **srHook** status is **SFAIL**, the local recovery takes place instead, and fencing is not triggered.

We recommend that you set **PREFER_SITE_TAKEOVER** to **true**. This allows the HA cluster to trigger a takeover when a failure of the primary HANA instance is detected. In most cases it takes less time for the new HANA primary instance to become fully active after a takeover, than it takes for the original primary instance to restart and reload all data back from disk into memory.

Leave **AUTOMATED_REGISTER** set to **false** to give the operator the option to first verify the health and data consistency of the previously failed primary instance, before you manually register it as the new secondary instance to re-establish the HANA system replication between both instances and manually start the instance.

Set **AUTOMATED_REGISTER** to **true** to enable the automatic registration of the former primary instance as the new secondary after a takeover occurs. This increases the availability of the HANA system replication setup and prevents so-called dual-primary situations in the SAP HANA system replication environment.

A.5. SAPHANAFILESYSTEM RESOURCE PARAMETERS

Parameters that are available for the configuration of **SAPHanaFilesystem** resources are shown below:

Resource options	Required	Default	Description
SID	yes		SAP system identifier.
InstanceNumber	yes		Number of the SAP HANA instance.
DIRECTORY	no	**/hana/shared/<SID>/**	Path to the directory to be monitored. The resource agent creates data in its own subdirectory **.heartbeat_SAPHanaFilesystem** inside the **DIRECTORY** path. Do not change this hidden subdirectory.
ON_FAIL_ACTION	no	fence	Action in case of a repeated monitor failure. • **ignore**: Do nothing, just report the failure into logs. • **fence**: Trigger stop failure and node fencing, if the required conditions are met.

APPENDIX B. USEFUL INFORMATION

B.1. EXPLAINING CLUSTER INFORMATION

When the cluster resources start they execute the first monitor operation and gather initial resource information. The HANA resources add node attributes and cluster properties for the collected landscape information, which describes the current state of SAP HANA databases on the cluster nodes.

Cluster status with node attributes

```
[root]# pcs status --full
...

Node List:
  * Node node1 (1): online, feature set 3.19.0
  * Node node2 (2): online, feature set 3.19.0

Full List of Resources:
  * Clone Set: cln_SAPHanaTop_RH1_HDB02 [rsc_SAPHanaTop_RH1_HDB02]:
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node2
    * rsc_SAPHanaTop_RH1_HDB02  (ocf:heartbeat:SAPHanaTopology):        Started node1
  * Clone Set: cln_SAPHanaCon_RH1_HDB02 [rsc_SAPHanaCon_RH1_HDB02] (promotable):
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Unpromoted node2
    * rsc_SAPHanaCon_RH1_HDB02  (ocf:heartbeat:SAPHanaController):      Promoted node1
  * Clone Set: cln_SAPHanaFil_RH1_HDB02 [rsc_SAPHanaFil_RH1_HDB02]:
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started node2
    * rsc_SAPHanaFil_RH1_HDB02  (ocf:heartbeat:SAPHanaFilesystem):      Started node1
  * rsc_vip_RH1_HDB02_primary   (ocf:heartbeat:IPaddr2):        Started node1
  * rsc_vip_RH1_HDB02_readonly  (ocf:heartbeat:IPaddr2):        Started node2

Node Attributes:
  * Node: node1 (1):
    * hana_rh1_clone_state              : PROMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC1
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.078.00
    * hana_rh1_vhost                    : node1
    * master-rsc_SAPHanaCon_RH1_HDB02   : 150
  * Node: node2 (2):
    * hana_rh1_clone_state              : DEMOTED
    * hana_rh1_roles                    : master1:master:worker:master
    * hana_rh1_site                     : DC2
    * hana_rh1_srah                     : -
    * hana_rh1_version                  : 2.00.078.00
    * hana_rh1_vhost                    : node2
    * master-rsc_SAPHanaCon_RH1_HDB02   : 100

...
```

Cluster properties

The new generation of SAP HANA resource agents stores information about the HANA instance as cluster properties in the property set named **SAPHanaSR**.

You can query the CIB (cluster information base) as the **root** user to check the content of the cluster attributes. The HANA resources and the **HanaSR** hook update these attributes.

Example:

```
[root]# cibadmin --query --xpath "//crm_config//cluster_property_set[@id='SAPHanaSR']"
<cluster_property_set id="SAPHanaSR">
  <nvpair id="SAPHanaSR-hana_rh1_site_srHook_DC2" name="hana_rh1_site_srHook_DC2"
value="SOK"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_mns_DC2" name="hana_rh1_site_mns_DC2"
value="node2"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_mns_DC1" name="hana_rh1_site_mns_DC1"
value="node1"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lpt_DC2" name="hana_rh1_site_lpt_DC2" value="30"/>
  <nvpair id="SAPHanaSR-hana_rh1_glob_sec" name="hana_rh1_glob_sec" value="DC2"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_opMode_DC2" name="hana_rh1_site_opMode_DC2"
value="logreplay"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srHook_DC1" name="hana_rh1_site_srHook_DC1"
value="PRIM"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lpt_DC1" name="hana_rh1_site_lpt_DC1"
value="1746780482"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srPoll_" name="hana_rh1_site_srPoll_" value="SOK"/>
  <nvpair id="SAPHanaSR-hana_rh1_glob_topology" name="hana_rh1_glob_topology"
value="ScaleUp"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lss_DC2" name="hana_rh1_site_lss_DC2" value="4"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_lss_DC1" name="hana_rh1_site_lss_DC1" value="4"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srr_DC2" name="hana_rh1_site_srr_DC2" value="S"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srr_DC1" name="hana_rh1_site_srr_DC1" value="P"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srMode_DC2" name="hana_rh1_site_srMode_DC2"
value="sync"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srMode_" name="hana_rh1_site_srMode_" value="sync"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srPoll_DC2" name="hana_rh1_site_srPoll_DC2"
value="SOK"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srMode_DC1" name="hana_rh1_site_srMode_DC1"
value="sync"/>
  <nvpair id="SAPHanaSR-hana_rh1_glob_prim" name="hana_rh1_glob_prim" value="DC1"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_srPoll_DC1" name="hana_rh1_site_srPoll_DC1"
value="PRIM"/>
  <nvpair id="SAPHanaSR-hana_rh1_site_opMode_DC1" name="hana_rh1_site_opMode_DC1"
value="logreplay"/>
</cluster_property_set>
```

SAPHanaSR-showAttr

You can use the tool **SAPHanaSR-showAttr** to display all of the HANA cluster attribute information in a preformatted overview.

Check this status in addition to **pcs status [--full]** to see the overall landscape health.

Example:

■

```
[root]# SAPHanaSR-showAttr
Global cib-update dcid prim sec sid topology
---------------------------------------------
global 0.550.2    2    DC1  DC2 RH1 ScaleUp

Resource                promotable
-----------------------------------
cln_SAPHanaCon_RH1_HDB02 true
cln_SAPHanaTop_RH1_HDB02

Site lpt        lss mns   opMode    srHook srMode srPoll srr
-------------------------------------------------------------
DC2  30         4   node2 logreplay SOK    sync   SOK    S
DC1  1746780164 4   node1 logreplay PRIM   sync   PRIM   P

Host  clone_state roles                      score site srah version     vhost
-----------------------------------------------------------------------------------
node1 PROMOTED    master1:master:worker:master 150   DC1  -    2.00.078.00 node1
node2 DEMOTED     master1:master:worker:master 100   DC2  -    2.00.078.00 node2
```